

LakeHouse Studio

Complete Architecture Specification

Build data lakes by shopping for components.

*End-to-end platform for assembling, installing, and operating
production-grade lakehouses on any infrastructure.*

Founding Technical Document

Version 1.0 · May 2026

Author: Vishnu · Hyderabad, India

Contents

1. Executive summary
2. Vision and problem statement
3. Architectural principles
4. System context
5. The six layers
6. The compatibility matrix in depth
7. The user journey from shop to query
8. The install workflow as a state machine
9. Data flow and security architecture
10. Deployment topology examples
11. Technology stack and justification
12. Phased roadmap (24 months)
13. Risks and mitigations
14. Glossary and references

1. Executive Summary

LakeHouse Studio is a plug-and-play platform that lets any data team assemble, validate, install, and operate a production-grade data lakehouse through a guided shopping experience. The user selects components — storage format, query engine, catalog, orchestrator, object storage, observability — adds them to a cart, provides server details, and the platform performs server inspection, compatibility validation, resource sizing, idempotent installation, end-to-end verification, and ongoing day-2 operations. What today takes six to eight weeks of integration engineering becomes a thirty-minute self-service workflow.

The platform is built on six clearly separated layers: a Presentation layer (web console, CLI, public API), an Orchestration layer (the controller and its workflow engine), an Intelligence layer (the compatibility solver and resource sizer), a Knowledge layer (the open-source compatibility matrix that constitutes the platform's defensible moat), an Execution layer (a small Go agent that runs on or near customer infrastructure), and the Target Infrastructure layer where the actual components live. The separation is deliberate: each layer has one job, can be replaced or extended independently, and obeys strict contract boundaries.

A foundational architectural commitment runs through every decision: **customer data never enters the LakeHouse Studio control plane**. The control plane manages metadata, workflow state, and audit logs. The data plane — query results, table contents, file storage — lives entirely within the customer's infrastructure. Communication between control plane and data plane is via outbound-initiated mTLS-secured gRPC from the agent. This separation enables compliance with India's DPDP Act 2023, the EU's GDPR, healthcare HIPAA equivalents, and air-gapped deployments for the most regulated environments.

The compatibility matrix is the platform's strategic asset. It is an open-source, Git-backed, community-contributable knowledge base of which component versions work together, what they require, and what they recommend at different scales. Every claim in the matrix is backed by CI test evidence — nothing is asserted without proof. This makes the matrix more than data; it is a trust contract with users.

The build plan covers 24 months across four parallel tracks: Track A (OSS core platform), Track B (consulting accelerator generating revenue from month four), Track C (managed SaaS deferred to month nine), and Track D (continuous matrix expansion). The plan is sized for a four-person team augmented by AI tooling, with explicit milestones and exit criteria at each phase.

This document specifies the architecture in full. It is the founding technical reference for the platform and is intended to be read by founders, engineers, design partners, and investors who need to understand what LakeHouse Studio is, how it works, and how it will be built.

2. Vision and Problem Statement

2.1 The Problem

Setting up a production data lakehouse today is a multi-month engineering project that demands deep expertise across at least six independent component categories. A data team must choose a storage format (Iceberg, Hudi, or Delta), a query engine (Trino, Spark, StarRocks, or others), a catalog (Nessie, Hive Metastore, Polaris), an orchestrator (Airflow, Dagster), an object storage layer (MinIO for on-prem or S3-compatible managed services), and an observability stack (typically Grafana + Prometheus). Each choice has dozens of compatible and incompatible version combinations with the others. A typical decision space contains over 200 viable component-version combinations, of which perhaps fifty have been actually tested together by anyone.

The integration work is non-trivial. Iceberg 1.4 works with Trino 435 but not with earlier Trino versions due to REST catalog API changes. Hudi 0.14 has Spark version constraints that differ from Hudi 0.15. The Hive Metastore protocol shifted between versions 3.x and 4.x. Nessie's catalog implementation has nuances with each engine. Engineers discover these incompatibilities the hard way, usually after multiple days of configuration changes and silent failures. The cost is significant: most mid-size companies spend four to eight weeks on lakehouse plumbing before touching a single byte of actual data.

For regulated industries — healthcare, financial services, government — the problem is worse. Cloud-managed offerings like Databricks SQL or Snowflake don't meet data residency requirements under India's DPDP Act 2023, the EU's GDPR, or sector-specific regulations like the RBI's data localisation mandates. These organisations need on-premise or sovereign-cloud deployments, which means even less managed support and even more integration burden.

The status quo for these teams is a choice between three bad options: invest in months of internal engineering to assemble a lakehouse, pay enterprise consulting firms hundreds of thousands of dollars for the same outcome, or settle for a less-capable single-vendor solution that compromises on flexibility or data sovereignty.

2.2 The Vision

LakeHouse Studio is the e-commerce checkout for data lakehouses. Users browse a catalog of components organised by category, with rich detail cards showing maturity, license, and compatibility. They add components to a cart, where real-time validation immediately flags conflicts and suggests resolutions. At checkout, they provide server details — SSH credentials, a kubeconfig, or AWS credentials — and the platform inspects the target environment, recommends resource allocation, performs a full pre-flight dry-run, and installs the entire stack

idempotently. The user sees live progress streaming in the console, and at the end has a working lakehouse with a demo dataset they can query immediately.

Beyond initial install, the platform manages the lakehouse for its operational lifetime: monitoring dashboards pre-wired and live, backup schedules running automatically, alert rules configured per component, upgrade paths suggested when compatible newer versions appear, and disaster recovery tested by the platform itself.

The product is opinionated where opinions add value — recommended stacks for healthcare, fintech, e-commerce — and open where flexibility matters: any valid component combination is installable, any deployment target is supported, any cloud or on-prem environment is welcome. The code is open source under Apache 2.0; the matrix is open source under Apache 2.0; users can self-host the entire platform if they want full sovereignty, or use a managed offering if they want zero infrastructure burden.

2.3 Target Users and Markets

The primary user is a data engineer or platform engineer at a company that needs a lakehouse but doesn't have the months of integration runway. This user persona is well-represented across:

- Mid-size Indian companies in regulated industries (healthcare, BFSI) where DPDP compliance forces on-premise or sovereign-cloud deployments and where cloud-managed alternatives are not viable.
- Cost-conscious startups and growth-stage companies who want lakehouse capabilities without the per-query pricing of managed cloud warehouses.
- Enterprise teams undertaking digital transformation and migrating from legacy warehouses (Teradata, Netezza, Oracle) to open formats but lacking the in-house expertise to assemble the new stack from scratch.
- Consulting and systems integrator firms who deliver data platform implementations to clients and want to standardise on a repeatable, validated stack.
- Academic institutions and research labs that need lakehouse infrastructure for data-intensive science but lack dedicated platform engineering teams.

The secondary user is the platform administrator — the person responsible for governance, RBAC, audit, and billing — who interacts primarily with org-level features rather than individual stacks. The tertiary user is the end consumer of the lakehouse: analysts, data scientists, ML engineers — whose experience LakeHouse Studio shapes indirectly by ensuring their lakehouse is fast, reliable, and well-monitored.

2.4 Why Now

Three convergent trends make LakeHouse Studio possible and necessary in 2026 that were not in 2020. First, the open table format wars have effectively settled around Iceberg, Hudi, and Delta — there is no longer a moving target on storage format. Second, Kubernetes operators for

data infrastructure have matured significantly; what required custom deployment automation three years ago is now mostly declarative Helm charts. Third, regulatory pressure for data sovereignty has intensified globally, making on-premise and sovereign-cloud deployments a growing requirement rather than a shrinking one.

On the demand side, the number of companies that need a lakehouse but cannot afford weeks of integration engineering has grown dramatically as analytics-driven decision-making has become table stakes across industries. The market is no longer just tech companies and finance; it now includes manufacturing, retail, healthcare, education, and government — all of whom face the same plumbing problem.

3. Architectural Principles

Every architectural decision in LakeHouse Studio derives from a small set of foundational principles. These are not slogans — each translates into specific design choices documented in the sections below. When tradeoffs arise, these principles guide resolution.

3.1 Customer Sovereignty

Customer data never enters the control plane. The platform stores stack specifications, install state, health metrics, and audit logs — never table data, query results, or schema contents that contain sensitive information. This principle is non-negotiable and drives the entire control-plane / data-plane separation described in Section 9.

3.2 Evidence-Backed Compatibility

Every compatibility claim in the matrix is backed by CI test evidence. Nothing is asserted without proof. When the matrix says Iceberg 1.5 works with Trino 440 and Nessie 0.77, there exists a CI run with a timestamp, an environment specification, and a test output showing that exact combination running real queries successfully. This evidence is public, audit-able, and reproducible.

3.3 Idempotent and Reversible Operations

Every operation can be re-run without ill effect, and every operation has a defined reverse operation. Install can be retried after partial failure without leaving the system in an inconsistent state. Every install has a corresponding uninstall that cleanly removes all artifacts. Upgrades have rollback paths. This is what makes the platform genuinely stable rather than fragile.

3.4 Outbound-Only Networking

The agent dials home; the control plane never reaches into customer networks. This works in restrictive environments — hospitals with strict firewalls, banks with no inbound exceptions, defense networks with air gaps. Customers do not need to open inbound ports, expose endpoints, or configure NAT traversal for LakeHouse Studio.

3.5 Open Source by Default

The core platform — console, controller, solver, agent, matrix — is open source under Apache 2.0. Customers can audit, fork, build from source, and self-host. Only the managed SaaS control plane code (when it exists) is source-available under BSL to prevent unfair commoditisation. This commitment is structural: the value proposition is the integrated experience and operational excellence, not opacity.

3.6 Separation of Concerns by Layer

Each architectural layer has exactly one responsibility. The Presentation layer renders. The Orchestration layer coordinates. The Intelligence layer computes. The Knowledge layer remembers. The Execution layer acts. The Target Infrastructure layer runs the actual data lakehouse. No layer reaches across boundaries to short-circuit work. This separation is what allows independent evolution of each piece over a decade.

3.7 Failure-Mode-First Design

Every workflow is designed first for failure, with success as the easy case. What happens when the agent loses connectivity mid-install? What happens when the catalog database is corrupted? What happens when an upstream component release breaks our matrix? Each of these has a defined recovery path documented and tested. Optimism is not an engineering strategy.

3.8 Honest Communication

Every failure produces an actionable diagnostic, never a raw stack trace. Every claim made by the platform has a confidence level and an explanation. Marketing and documentation align with engineering reality: we say tested combinations rather than any combination, and we publish the test matrix so users can verify.

3.9 Performance as a Feature

User-facing operations have explicit performance budgets. Compatibility validation must run in under 100 milliseconds for keystroke-level UI responsiveness. Server inspection completes in under 30 seconds. Demo install finishes in under 30 minutes on a 32 GB VM. These budgets shape architecture: they are why the solver is in Rust, why the agent is a single static binary, and why the matrix has a runtime-optimised PostgreSQL mirror separate from its Git source-of-truth.

3.10 Extensibility through Contracts

New components, new deployment targets, new monitoring backends should be addable without changing core architecture. The matrix schema, the agent protocol, and the playbook structure are versioned contracts. Community contributions of new component support follow the same well-defined path the core team uses — there is no privileged code path.

4. System Context

Before drilling into internal architecture, the system context shows how LakeHouse Studio fits into the broader ecosystem of users, infrastructure, and integrations. The diagram below identifies every external actor and integration point.

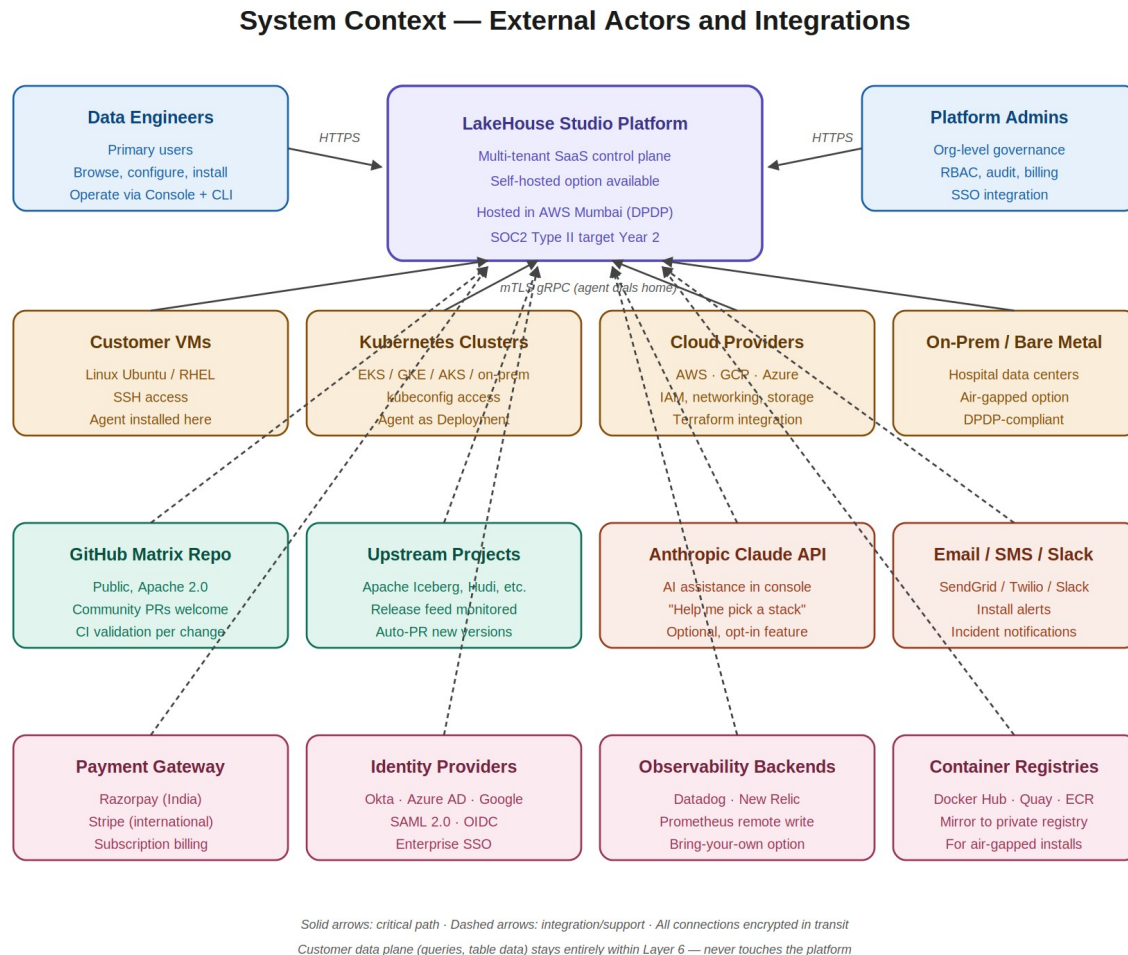


Figure 1 — System context showing all external actors, infrastructure targets, and third-party integrations.

4.1 External Actors

Two primary user populations interact directly with the platform. **Data engineers** are the platform's primary users; they browse the catalog, configure stacks, perform installs, and operate the deployed lakehouse through the Console and CLI. **Platform administrators** handle org-level governance: managing user accounts, configuring SSO, defining RBAC roles, reviewing audit logs, managing billing, and setting platform-wide policies.

4.2 Infrastructure Targets

LakeHouse Studio operates against four broad classes of target infrastructure, each with its own deployment characteristics:

- Linux VMs accessed via SSH — the most flexible target, suitable for cloud (EC2, GCE, Azure VMs), on-premise virtualisation (VMware, Proxmox), and bare metal. Agent runs as a systemd service.
- Kubernetes clusters accessed via kubeconfig — for cloud-managed (EKS, GKE, AKS) and on-premise (OpenShift, Rancher, plain kubeadm). Agent runs as a Deployment in a dedicated namespace.
- Cloud provider APIs (AWS, GCP, Azure) — for provisioning the underlying infrastructure when the customer wants the platform to handle that too. Uses Terraform/Pulumi under the hood.
- On-premise bare metal and air-gapped environments — for hospitals, defense, government, BFSI. Agent runs natively; matrix updates delivered via signed offline archives.

4.3 Third-Party Integrations

The platform integrates with several external services to deliver its full functionality:

- GitHub — for hosting the public matrix repository and accepting community pull requests.
- Upstream open-source projects (Apache Iceberg, Apache Hudi, Delta Lake, etc.) — monitored for new releases; their release feeds trigger automated PRs to add new versions to our matrix.
- Anthropic Claude API — optional, opt-in AI assistance feature in the console for stack recommendation and troubleshooting help.
- Notification providers — SendGrid for email, Twilio for SMS, Slack and Microsoft Teams for chat-based install alerts and incident notifications.
- Payment gateways — Razorpay for the Indian market, Stripe internationally, for subscription billing.
- Identity providers — Okta, Azure AD, Google Workspace via SAML 2.0 or OIDC for enterprise SSO.
- Observability backends — Datadog, New Relic, customer Prometheus instances via remote-write for organisations that want to consolidate metrics in their existing tooling.
- Container registries — Docker Hub, Quay, AWS ECR, with mirror-to-private-registry support for air-gapped customers.

4.4 Data Flow Boundaries

A critical aspect of the system context is what does **not** cross boundaries. Customer query results, table data, and file contents never traverse the connection between the agent and the controller. The agent reports stack health (component status, resource utilisation, latency percentiles) and install state (task progress, error messages with sensitive content redacted),

but never the contents of the customer's tables. This is enforced at the protocol level in the agent: the gRPC service definitions simply have no fields capable of carrying table data.

5. The Six Architectural Layers

LakeHouse Studio is structured as six clearly separated layers, each with a single responsibility, each independently developable and deployable. The diagram below summarises all six; the subsections that follow specify each in detail.

LakeHouse Studio — Six-Layer Architecture

L1 — Presentation Layer

Web Console (Next.js 14) · CLI (Go binary "lks") · Public REST API (OpenAPI 3.1)
Shopping cart UX · Component catalog · Server dashboard · Install wizard · Monitoring
Single entry point for humans and automation. Replaceable surfaces.

L2 — Orchestration Layer

Controller (Python 3.12 FastAPI) · Celery + Redis job queue · PostgreSQL 16 state store
Stack assembly · Install workflow DAG · Tenant management · Audit log · Notifications
The brain. Source of truth for workflow state.

L3 — Intelligence Layer

Solver (Rust CDCL SAT engine + PyO3) · Resource sizer (Python) · ML recommender (deferred)
Compatibility resolution · Resource allocation · Stack recommendations
Pure functions. Stateless. Called by orchestration layer.

L4 — Knowledge Layer (The Moat)

Matrix store (Git-backed YAML + PostgreSQL mirror) · Test evidence archive
Component specs · Compatibility constraints · Resource profiles · Migration paths
Versioned, community-contributable, every claim backed by CI evidence.

L5 — Execution Layer

Agent (Go static binary, ~30MB, no runtime deps) · Ansible · Helm · Docker Compose
Server inspection · Idempotent installs · Health checks · Lifecycle ops · Log streaming
Outbound mTLS gRPC only. Runs on customer infrastructure.

L6 — Target Infrastructure (Customer Choice)

Deployment: Linux VMs (SSH) · Kubernetes (kubeconfig) · Docker Compose · Bare metal
Cloud: AWS · GCP · Azure · On-prem · Hospital data centers · Air-gapped environments
Components: Iceberg, Hudi, Delta · Trino, Spark, StarRocks · Airflow, Dagster · Nessie, HMS
MinIO, S3, GCS, ADLS · Grafana, Prometheus, Loki
Customer-owned. Customer-controlled. Data never leaves this layer.

Figure 2 — The six architectural layers and what each one does.

5.1 Layer 1 — Presentation

The Presentation Layer's job is to make a complex technical decision feel as simple as ordering groceries. It is built from three surfaces that share a common API: the web Console, the command-line interface (CLI), and the public REST API.

5.1.1 The Console

The Console is a Next.js 14 application running React Server Components for fast initial page loads, Tailwind CSS with shadcn/ui for the design system, and TypeScript end-to-end. It implements four primary views.

The Catalog is where users browse components organised by category. Each category — storage formats, query engines, catalogs, orchestrators, object storage, observability — has its own filterable, sortable list. Components are displayed as rich detail cards showing version, license, maintainer, maturity level (GA/Beta/Experimental), key technical specifications, and a compatibility score computed live against whatever is already in the cart.

The Cart is a persistent right-hand drawer showing selected components with real-time compatibility validation. As users add or remove items, the compatibility solver runs and the drawer updates with green checks (fully compatible), yellow warnings (compatible with caveats), or red errors (incompatible). Each non-green status comes with an explanation and a one-click remediation: Trino 430 is too old for Iceberg 1.5, switch to Trino 440.

The Inspector view appears after the user enters server credentials. It shows a live dashboard of detected resources — CPU model and cores, RAM total and available, disk per mount, OS distribution, network reachability, installed dependencies. Sliders let the user allocate memory to each component, with recommendations from the resource sizer pre-populated.

The Install Console is a live-streaming view of installation progress. Each component has its own collapsible log panel; the active task's panel is auto-expanded; completed tasks show green checkmarks; failed tasks show red Xs with diagnostic detail and retry/skip/abort options.

5.1.2 The CLI

The CLI is a Go binary called `lks` (LakeHouse Studio) that mirrors every Console operation: `lks catalog list`, `lks cart add iceberg@1.5`, `lks install --server prod-vm-01`. The CLI is not a thin wrapper — it talks to the same REST API the Console uses, with output formatting (table, JSON, YAML) suited to terminal use. This is critical because data engineers will want to script lakehouse provisioning into their existing Terraform or Ansible workflows.

The CLI supports interactive flows for human use and fully non-interactive flows for automation. A typical script might be:

```
lks cart create --name production-stack
lks cart add iceberg@1.5 trino@440 nessie@0.77 minio@latest
```

```
lks cart validate
lks install --server prod-vm-01 --confirm
lks verify production-stack
```

Every CLI command returns structured JSON when invoked with `--output json`, making it CI/CD friendly.

5.1.3 The Public API

The Public API is OpenAPI 3.1 specified, versioned (`/v1/...`), and authenticates via API keys or OAuth 2.0. Every operation in the Console and CLI is available via this API, which means customers can build their own internal tooling on top.

The API is designed RESTfully where REST makes sense (resource CRUD) and uses WebSockets for real-time streams (install progress, log tailing). It includes:

- Catalog endpoints: list components, get component details, query compatibility
- Cart endpoints: create, modify, validate, clear
- Server endpoints: register, inspect, list, deregister
- Install endpoints: dry-run, execute, abort, rollback, status
- Operations endpoints: backup, restore, upgrade, scale, uninstall
- Observability endpoints: stack health, component metrics, query stats
- Administration endpoints: users, roles, API keys, audit log

5.2 Layer 2 — Orchestration

The Orchestration Layer is the brain. It owns the workflow state, mediates between user intent and execution, and is the only layer that talks to the database. It is a Python 3.12 FastAPI application backed by PostgreSQL 16 for transactional state and Redis 7 for the Celery task queue.

5.2.1 Six Subsystems

The API Gateway terminates HTTP, handles authentication (JWT validation for users, API key validation for programmatic access, mTLS certificate validation for agent connections), applies rate limiting, and routes requests to the appropriate handler. All inbound traffic goes through this gateway. Rate limits are tiered by user plan and apply both globally and per-endpoint.

The Stack Assembler takes a user's cart contents and produces a fully-resolved stack specification. It calls out to the Intelligence Layer's solver to fill in transitive dependencies (Iceberg requires a catalog, so add Nessie unless something else is in the cart) and resolve version ranges (cart says `trino@*`, resolve to `trino@440`). The output is a complete, validated stack spec ready for installation.

The Install Orchestrator breaks an install into a directed acyclic graph (DAG) of tasks. For a typical stack the DAG looks like: provision storage backend, then start catalog, then register catalog in storage, then start query engine, then start orchestrator, then start observability, then load demo dataset, then run smoke tests, then mark ready. Each task is independently retrievable and rollback-able. The orchestrator dispatches tasks to the appropriate agent, waits for completion, and proceeds to the next task in topological order. Tasks that have no inter-dependency run in parallel.

The Tenant Manager handles multi-tenancy: organisations, users, roles, API keys, billing accounts. It enforces tenant isolation at every layer — every database query is parameterised by tenant ID; every API endpoint validates that the requester has access to the requested resource; every audit log entry is tagged with the tenant.

The Event Log captures every state transition for debugging and compliance. Every API call, every install task start and complete, every agent communication, every error — all are written to an append-only event log table. This log is the source of truth for the audit feature and for forensic analysis when things go wrong.

The Notification Service pushes WebSocket updates to the Console so install progress feels live, sends email/SMS/Slack messages for important state changes, and handles user-configured alert rules. It uses Redis pub/sub internally for decoupled fan-out.

5.2.2 Why Python and FastAPI

Python was chosen for the controller because the team has deep Python expertise, the matrix engine requires sophisticated graph algorithms where Python's ecosystem (NetworkX, packaging, Pydantic) is unmatched, Ansible — which the agent uses — is Python-native, and future ML integrations (the recommender) are easier in Python. FastAPI specifically because it provides automatic OpenAPI documentation, native async support for WebSocket and gRPC bridging, and excellent performance (10K+ requests/second per worker).

5.2.3 Hosting Model

A critical design choice: the controller never SSHes into a customer server directly. All execution flows through the agent. This means the controller can run in a SaaS-hosted environment while the agent runs in customer-controlled networks, satisfying data residency requirements without requiring the entire control plane to be customer-hosted. For customers who need full air-gapped deployment, the entire controller can also be self-hosted using the same container images.

5.3 Layer 3 — Intelligence

The Intelligence Layer answers three questions: Will these components work together? How much resource should each component get? Given what the user wants, what should we recommend? It is composed of three modules, each with distinct concerns.

5.3.1 The Solver

The Solver is a constraint-satisfaction engine written in Rust and exposed to Python via PyO3 bindings. It models the compatibility problem as a SAT (Boolean satisfiability) instance — each component-version pair is a Boolean variable, each compatibility constraint is a clause — and uses a CDCL (conflict-driven clause learning) algorithm to find solutions efficiently.

Why Rust and CDCL? Because at scale this matters. With 200 components across an average of 5 versions each and pairwise constraints between them, the naive search space is intractable. CDCL learns from each conflict it encounters, pruning the search dramatically. Cargo, Pub, and apt all use variants of this approach for the same reason. For the V1 matrix with 30 components, the solver runs in microseconds; the Rust investment pays off when the matrix grows past 100 entries.

The solver is invoked at multiple points: live as the user adds items to the cart (validation), during checkout (final resolution), at install time (final lock), and during upgrade planning (compatibility re-check). It is stateless and horizontally scalable.

Solver — Compatibility Resolution Algorithm

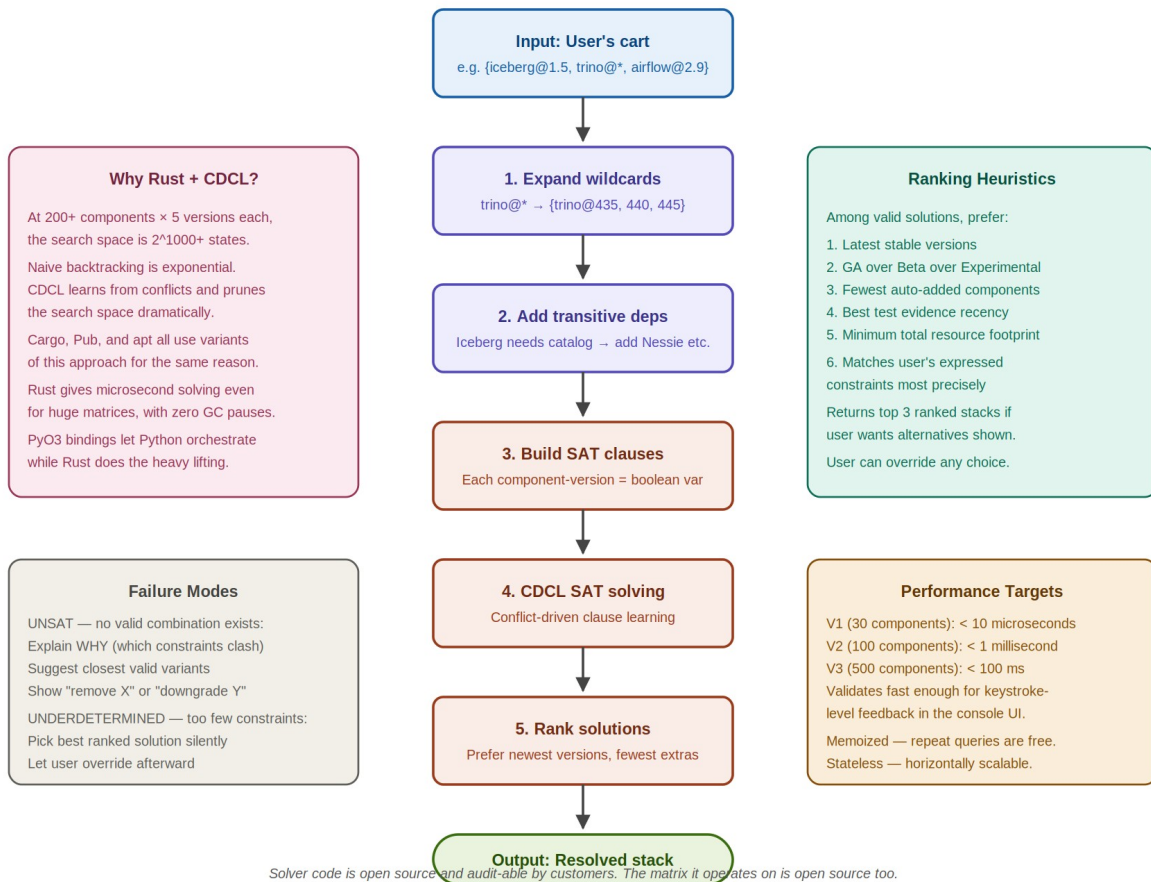


Figure 3 — Solver algorithm flow from cart input through CDCL solving to ranked output.

5.3.2 The Resource Sizer

The Resource Sizer is a Python module that takes a stack specification, a server inspection result, and a target workload size, and produces a recommended allocation: how much heap for Trino, how much memory for Spark executors, how much disk for Iceberg metadata, how many CPU cores for each worker. The recommendations come from three sources: vendor-published sizing guidance (Trino's documented memory model), our own benchmark results captured in the matrix, and configurable safety margins.

The sizer outputs structured recommendations the Console renders as pre-populated sliders. Users can override any recommendation; the sizer re-checks feasibility against the inspected resources and flags problems before the install starts.

5.3.3 The Recommender

The Recommender is the future: a Python ML service that learns from successful deployments. Given a description of the customer's need (I'm a 50-person fintech doing 10TB of analytics,

mostly batch with some interactive), it suggests a starting stack with rationale. This is the tell me what I need experience for users who don't know exactly what components to pick.

This sits behind a feature flag and only activates once we have enough deployment data to make recommendations useful — projected for month twelve. Initial implementation will use a retrieval-augmented generation (RAG) approach: maintain a corpus of stack pattern to use case mappings curated from real deployments, and use embeddings to find the closest match for new queries.

5.4 Layer 4 — Knowledge (The Moat)

The Knowledge Layer holds the compatibility matrix — the platform's most important strategic asset. The diagram below shows its internal structure; Section 6 covers it in even greater depth.

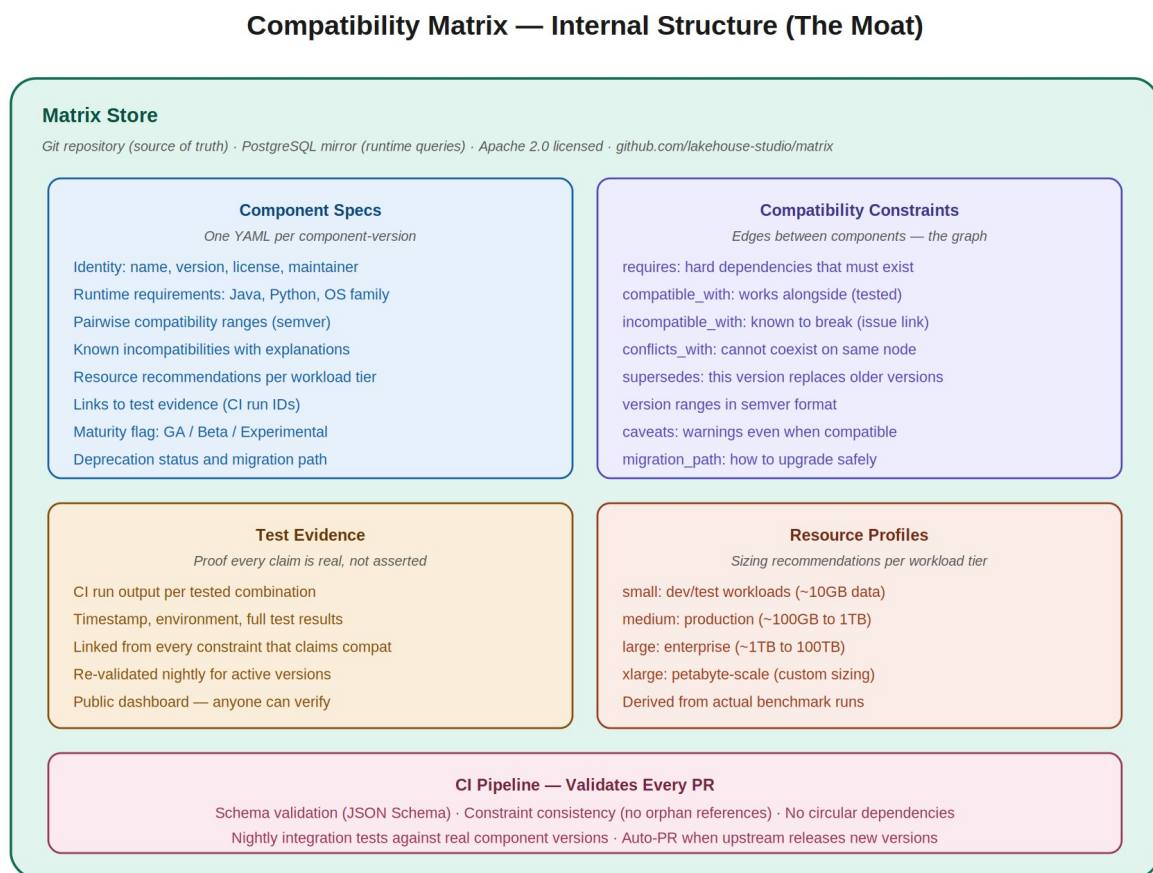


Figure 4 — The Knowledge Layer (compatibility matrix) internal structure.

5.4.1 Why the Matrix Is the Moat

Everything else in LakeHouse Studio can be copied or replicated by a determined competitor in months. A UI is a UI. A workflow engine is a workflow engine. The matrix — comprehensive,

evidence-backed, continuously maintained — takes years to build and even longer to gain community trust. Once trust is established, network effects compound: more contributors means more matrix entries means more users means more contributors. This is the same dynamic that made Linux distributions defensible despite being open source.

5.4.2 Architecture

The matrix is a Git repository at its core. Every change is a pull request, every entry has commit history, the community can fork and contribute. This is deliberate — it builds trust through transparency, enables external validation, and creates a flywheel where adoption improves the matrix.

The matrix mirrors to PostgreSQL for fast queries at runtime. The controller never reads YAML files directly during request handling; it queries the Postgres mirror which is updated by a CI job whenever the Git repo changes. This separates the human-editable source of truth (Git/YAML) from the machine-optimised read store (Postgres), giving both readability and performance.

5.4.3 Component Specifications

Each component-version is described in a YAML file with this structure:

- Identity: name, version, license, maintainer, source URL
- Runtime requirements: Java version range, Python version range, OS family, kernel version, libc version
- Compatible-with: pairwise compatibility rules with version ranges and evidence references
- Incompatible-with: known broken combinations with explanations and bug tracker links
- Conflicts-with: components that cannot coexist on the same node (e.g., port conflicts)
- Resource recommendations: minimum, recommended, and optimal allocations per workload tier
- Maturity flag: GA, Beta, or Experimental
- Deprecation status and migration path to successor versions
- Test evidence: links to CI runs proving every claimed compatibility

5.4.4 Maintenance Process

The matrix is maintained through a defined process. Upstream component releases trigger automated detection (a CI job watches release feeds for every tracked project). New versions get auto-PRd with placeholder spec files. Engineers and AI agents fill in the spec details, run validation tests, and merge. The whole pipeline is designed to keep up with Iceberg's 6-8 week release cadence and Trino's 2-week cadence.

5.5 Layer 5 — Execution

The Execution Layer is where strategy meets reality. The Agent is a Go static binary (approximately 30MB, no runtime dependencies) that runs on or near the target infrastructure. It

is what actually performs server inspection, installation, monitoring, and lifecycle operations. The diagram below shows its internal module structure.

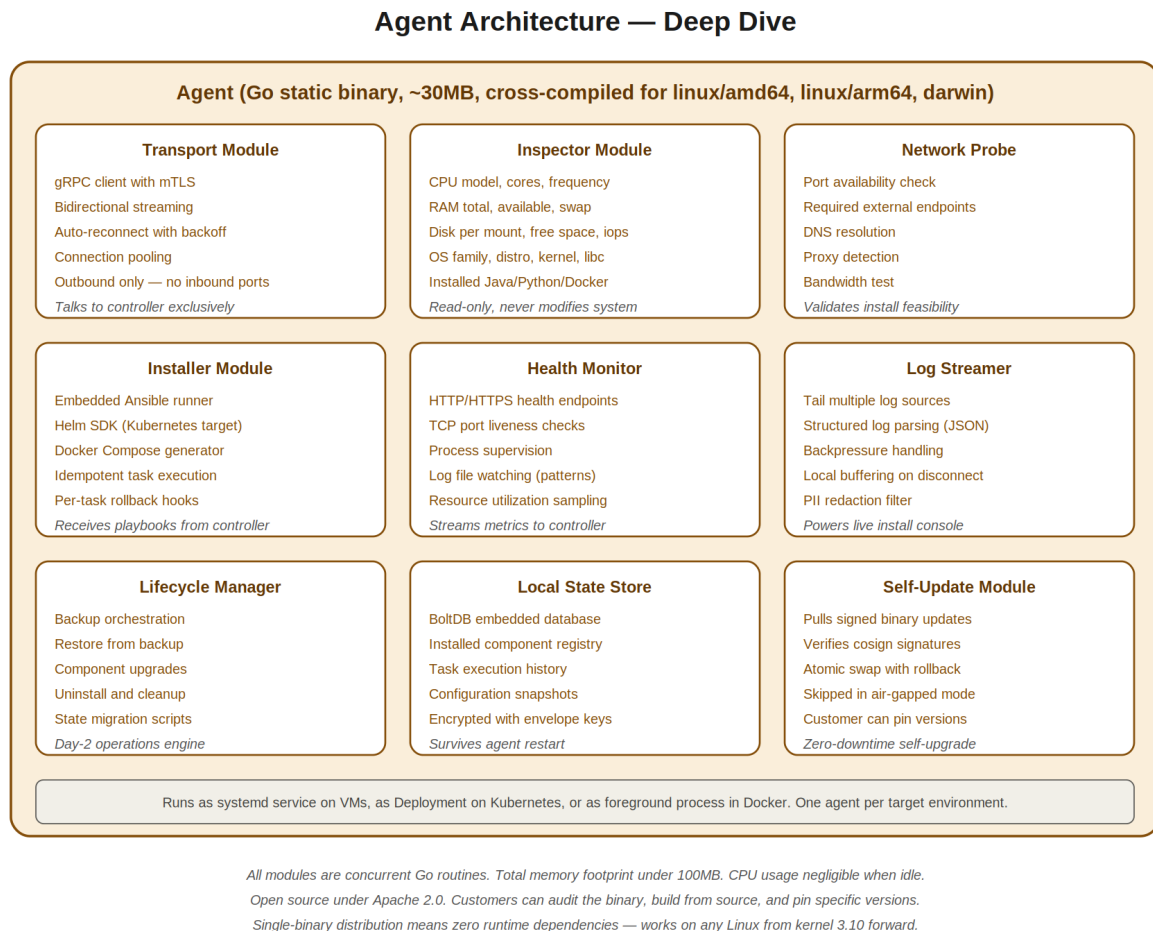


Figure 5 — Agent internal architecture showing all nine modules.

5.5.1 Why a Single Static Binary

The agent ships as a single static binary with no runtime dependencies. This choice matters because the agent must run on customer servers that we don't control. We cannot assume Python is installed, or Node, or even a specific glibc version. A Go static binary works on any Linux from kernel 3.10 forward (i.e., every supported Linux distribution from the last decade). It also makes air-gapped distribution trivial: copy one file, it runs.

5.5.2 The Nine Modules

Transport implements the gRPC client with mTLS, bidirectional streaming for log and metric streams, automatic reconnection with exponential backoff, and connection pooling. Critically, it only opens outbound connections — the agent never listens for inbound traffic.

Inspector discovers the target environment: CPU model and cores, RAM and swap, disk per mount with free space and IOPS estimates, OS family and version, kernel and libc, installed Java/Python/Docker versions. It is read-only and never modifies system state.

Network Probe validates network feasibility: port availability for the components being installed, reachability of required external endpoints (container registries, Maven Central, PyPI), DNS resolution, proxy detection, and bandwidth tests.

Installer is the workhorse. It runs Ansible playbooks for VM targets, applies Helm charts for Kubernetes targets, and generates/runs Docker Compose for local targets. All operations are idempotent; the same playbook can be applied multiple times safely. Each task registers a rollback hook so partial installs can be cleanly reverted.

Health Monitor runs continuous checks on installed components: HTTP/HTTPS health endpoints, TCP port liveness, process supervision, log file parsing for known error patterns, resource utilisation sampling. Metrics are streamed to the controller for the observability dashboards.

Log Streamer tails multiple log sources (system logs, component-specific logs, custom log paths), parses structured logs, handles backpressure when the controller is slow to consume, buffers locally on disconnect, and filters PII through a configurable redaction rule set.

Lifecycle Manager handles day-2 operations: backup orchestration, restore from backup, component upgrades, uninstall and cleanup, and state migration scripts. This is the module that makes the platform genuinely operational rather than install-only.

Local State Store uses BoltDB (an embedded key-value store, single file, no external dependencies) to persist the installed component registry, task execution history, and configuration snapshots. The store is encrypted with an envelope-encryption scheme; the master key lives only in memory and is fetched on agent start from the controller.

Self-Update allows the agent to pull signed binary updates from the controller, verify them with cosign signatures, atomically swap the running binary with rollback capability, and resume operation seamlessly. In air-gapped mode this is disabled; agents are updated manually with signed offline archives.

5.5.3 Deployment Models

The agent runs as a systemd service on Linux VMs, as a Kubernetes Deployment (replicas=1) on Kubernetes targets, and as a foreground process inside Docker for local development. One agent per target environment is sufficient; the agent coordinates multi-node operations from a single location.

5.6 Layer 6 — Target Infrastructure

The Target Infrastructure Layer is where the actual data lakehouse runs. This layer is entirely customer-owned and customer-controlled — LakeHouse Studio does not run components for the customer, it just installs and operates them on the customer’s infrastructure.

Components in this layer fall into six categories. Briefly: storage formats (Iceberg, Hudi, Delta) hold the actual table data; query engines (Trino, Spark, StarRocks) execute SQL against that data; catalogs (Nessie, Hive Metastore, Polaris) hold table metadata and schema; orchestrators (Airflow, Dagster) schedule data pipelines; object storage (MinIO, S3, GCS, ADLS) provides the underlying file storage; observability stacks (Grafana, Prometheus, Loki) provide monitoring and logging.

5.6.1 The Full Component Catalog

The catalog below is what users see in the Console. It is the V1 catalog; new entries are added continuously through the matrix maintenance process.

Component Catalog — What Users Can Shop For		
Storage Formats Apache Iceberg (1.4, 1.5, 1.6) Apache Hudi (0.14, 0.15) Delta Lake (3.0, 3.1, 3.2) <i>Choice depends on: write patterns (Hudi for upserts), ecosystem fit (Iceberg for vendor-neutral), and Databricks integration needs (Delta).</i>	Query Engines Trino (435, 440, 445) Apache Spark SQL (3.4, 3.5) StarRocks (3.1, 3.2, 3.3) <i>Trino for interactive analytics, Spark for batch and ML, StarRocks for sub-second OLAP with materialized views.</i>	Catalogs Project Nessie (0.77+) Hive Metastore (3.1, 4.0) Apache Polaris (0.1+) <i>Nessie for Git-like data branching, HMS for battle-tested compatibility, Polaris for multi-engine REST catalog standard.</i>
Orchestrators Apache Airflow (2.8, 2.9) Dagster (1.6, 1.7) Prefect (2.x) — V2 <i>Airflow for mature ecosystem and operator familiarity, Dagster for software-defined assets and superior local dev experience.</i>	Object Storage MinIO (latest) AWS S3 (managed) GCS, ADLS, Ceph S3 <i>MinIO for on-prem and air-gapped, cloud S3 for managed offerings, all S3-compatible backends work transparently.</i>	Observability Grafana (10.x, 11.x) Prometheus (2.50+) Loki (3.x) for logs <i>Pre-wired dashboards per stack combination. Bundled together by default but can be swapped for existing customer stacks.</i>
Auxiliary Components (Auto-Included) PostgreSQL — metadata backing for HMS, Airflow, Nessie Redis — caching layer for query plans NGINX — reverse proxy and TLS termination Cert-manager / Let's Encrypt — certificate lifecycle Vault — secrets management for the deployed stack <i>These are added automatically when required by user choices.</i>		Future Components (Roadmap) Apache Kafka — for streaming ingestion Debezium — CDC from operational databases dbt Core — transformation layer Apache Superset — BI and visualization OpenLineage / Marquez — data lineage tracking <i>Added per quarter based on user demand and matrix maturity.</i>

Total V1 catalog: 6 categories, ~25 component-versions, ~150 valid permutations
Each entry in the catalog maps to a Knowledge Layer YAML spec with test evidence

Figure 6 — Full V1 component catalog organised by category.

5.6.2 Auxiliary Components

Some components are auto-included by the platform when other choices require them. These include PostgreSQL (metadata backing for Hive Metastore, Airflow, Nessie), Redis (caching layer for query plans), NGINX (reverse proxy and TLS termination), cert-manager and Lets Encrypt (certificate lifecycle), and Vault (secrets management for the deployed stack). Users don't need to think about these — they appear automatically in the install plan when triggered by primary choices.

5.6.3 Future Components on the Roadmap

Several components are planned for future quarterly releases based on user demand and matrix maturity. These include Apache Kafka for streaming ingestion, Debezium for change data capture from operational databases, dbt Core as the transformation layer, Apache Superset for BI and visualisation, and OpenLineage/Marquez for data lineage tracking. Each addition follows the standard matrix expansion process: schema entry, test evidence, documentation, public release.

6. The Compatibility Matrix in Depth

Section 5.4 introduced the matrix as the platform's moat. This section specifies it in full, because the matrix is so central that getting it right is more important than getting any other component right.

6.1 Matrix Schema

Every matrix entry is a YAML file conforming to a strict JSON Schema. The schema is versioned (matrix-schema-v1, matrix-schema-v2, etc.) and the CI pipeline validates every PR against the current schema. The schema includes:

6.1.1 Identity Fields

- `component`: the canonical short name (e.g., iceberg, trino)
- `version`: the exact version string (e.g., 1.5.0, 440)
- `display_name`: human-friendly name for the UI (e.g., Apache Iceberg)
- `category`: one of storage, query, catalog, orchestrator, storage_backend, observability, auxiliary
- `license`: SPDX identifier (e.g., Apache-2.0)
- `maintainer`: organisation or community responsible
- `source_url`: link to the project's home
- `release_date`: when this version was released upstream
- `added_to_matrix`: when we added it to our matrix

6.1.2 Requirement Fields

- `requires.java`: SemVer range (e.g., >=11,<22) or null
- `requires.python`: SemVer range or null
- `requires.os_family`: array of supported families (linux, darwin)
- `requires.os_versions`: per-distro version ranges
- `requires.kernel`: minimum kernel version
- `requires.libc`: minimum glibc version
- `requires.cpu_arch`: array (amd64, arm64)
- `requires.ports`: list of ports the component needs to bind to

6.1.3 Compatibility Fields

Each compatibility entry specifies a target component, a version range, evidence, and any caveats:

- `compatible_with`: array of compatibility tuples
- Each tuple: {name, versions, evidence, caveats, maturity}
- `versions` uses SemVer range syntax: ">=435,<450"
- `evidence`: CI run identifier (e.g., test-run-2026-04-12-1428)

- caveats: free-text warnings (e.g., No support for V2 row-level deletes)
- maturity: ga, beta, or experimental for this specific pairing

6.1.4 Incompatibility Fields

Incompatibility entries explicitly document combinations that don't work, which is just as important as documenting what does:

- incompatible_with: array of incompatibility tuples
- Each tuple: {name, versions, reason, issue_link, workaround}
- reason: human-readable explanation
- issue_link: link to upstream issue tracker or our own analysis
- workaround: how to use the components anyway (e.g., downgrade X or use bridge component Y)

6.1.5 Resource Profile Fields

Resource profiles capture our tested sizing recommendations across workload tiers:

- resource_profiles: object keyed by tier (small, medium, large, xlarge)
- Each tier: {ram_gb, cpu_cores, disk_gb, network_mbps, notes}
- Profiles derived from documented benchmark runs
- Benchmark methodology published alongside the matrix

6.2 Sample Matrix Entry

Below is an annotated example of an Iceberg 1.5.0 matrix entry. The actual YAML is more verbose; this is the conceptual structure:

```
component: iceberg
version: 1.5.0
display_name: Apache Iceberg
category: storage
license: Apache-2.0
maintainer: Apache Software Foundation
requires:
  java: ">=11,<22"
  hadoop: ">=3.3.0,<3.4.0"
compatible_with:
  query_engines:
    - name: trino
```

```

    versions: ">=435,<450"
    evidence: test-run-2026-04-12-1428
    maturity: ga
  - name: spark
    versions: ">=3.4.0,<3.6.0"
    evidence: test-run-2026-04-14-0930
    maturity: ga
catalogs:
  - name: nessie
    versions: ">=0.77.0"
    evidence: test-run-2026-04-10-1015
    maturity: ga
incompatible_with:
  - name: trino
    versions: "<435"
    reason: REST catalog API mismatch
    issue_link: https://github.com/trinodb/trino/issues/XXXXX
resource_profiles:
  small: {ram_gb: 4, cpu_cores: 2, disk_gb: 50}
  medium: {ram_gb: 16, cpu_cores: 8, disk_gb: 500}
  large: {ram_gb: 64, cpu_cores: 32, disk_gb: 5000}

```

6.3 Test Evidence

Every claim in the matrix references a test evidence ID. The CI system maintains an immutable archive of test runs, each with:

- Timestamp of execution
- Complete environment specification (OS, Java version, container images used)
- Full test script and parameters
- Raw test output (logs, query results, performance numbers)
- Pass/fail determination
- Reviewer signoff (for non-automated tests)

Anyone can browse the evidence archive at the public dashboard and verify our claims independently. This transparency is what makes the matrix trustworthy.

6.4 Nightly Validation

Every night, the CI system re-runs validation tests for the top-50 most-used component combinations. This catches regressions when upstream components ship breaking changes, infrastructure drifts, or test environments evolve. Failed validations trigger automated alerts to the matrix maintenance team and — if the failure is severe — may temporarily mark the affected combinations as suspended until investigated.

6.5 Community Contribution Model

The matrix is open source under Apache 2.0 and lives at github.com/lakehouse-studio/matrix. Anyone can fork, propose changes, or contribute new entries via pull requests. The contribution process is:

1. Fork the matrix repository
2. Create a new component YAML or modify an existing one
3. Provide CI test evidence by running the test suite locally and submitting results
4. Submit a pull request with a description of the change
5. Automated CI runs schema validation, consistency checks, and re-validation tests
6. Core team or trusted community maintainers review and merge

Trusted contributors are granted commit access after a track record of quality contributions. The Apache Way of governance is the model: meritocracy, transparency, and community.

7. The User Journey from Shop to Query

Section 4 introduced the journey at a high level. This section walks through each step in detail, documenting the exact user experience and what the platform does behind the scenes.

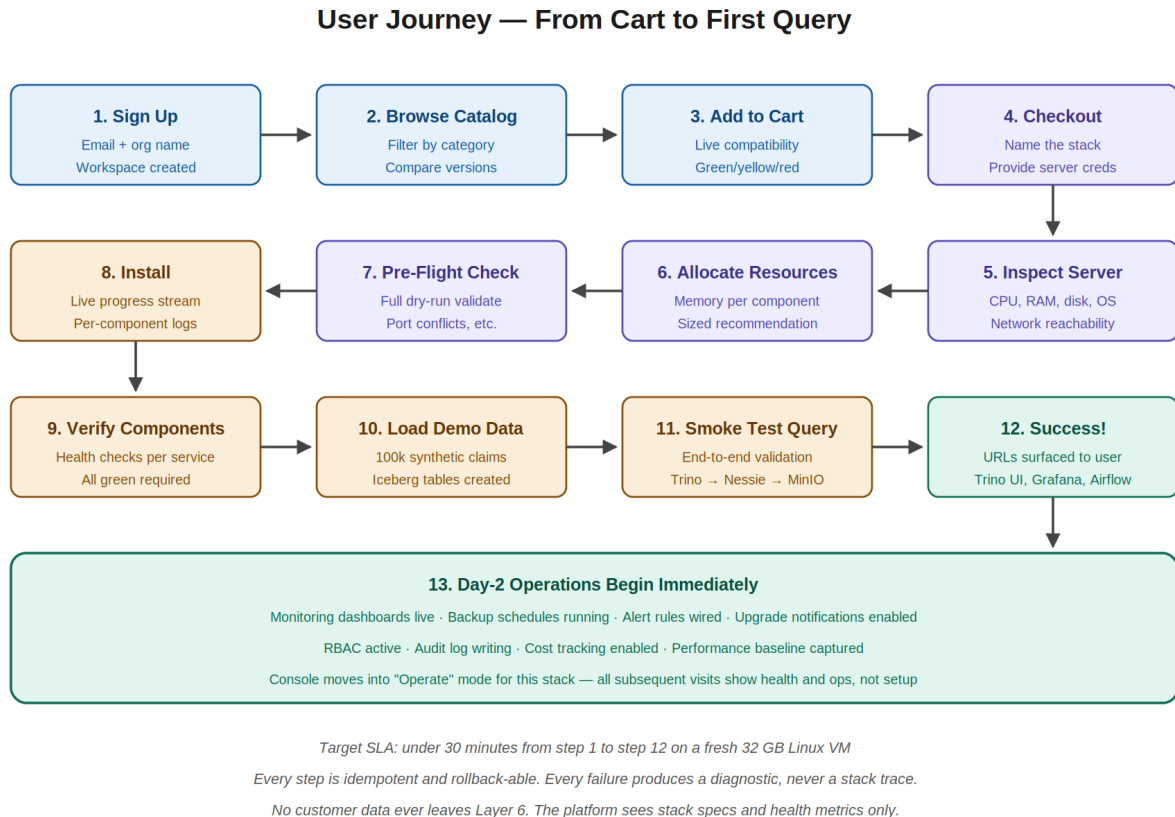


Figure 7 — The 13-step user journey from sign-up to operational lakehouse.

7.1 Step 1: Sign Up

A new user lands on the Console marketing page, clicks Get Started, and creates an account with email and organisation name. The system creates an organisation tenant, assigns the user as Owner, generates an initial API key (shown once and stored encrypted), and provisions a default workspace. For enterprise customers, SSO can be configured at this step instead of email/password.

7.2 Step 2: Browse Catalog

The Catalog opens to a category overview showing storage formats, query engines, catalogs, orchestrators, storage backends, and observability. Each category shows the components available with version counts and short descriptions. Users can filter by maturity (GA only /

include Beta / include Experimental), license (any / Apache 2.0 only / no GPL), and capability (supports streaming / supports row-level updates / etc.).

Clicking a component opens a detail page with the full specification: requirements, compatibilities, resource profiles, links to documentation, and links to the matrix entry on GitHub. Users can see exactly what they're committing to before adding to cart.

7.3 Step 3: Add to Cart

The cart is a persistent drawer. Adding a component triggers a solver run — visible to the user as a 100ms validation that returns instantly. The cart shows each item with version, a green/yellow/red status, and any caveats. As items are added, transitive dependencies are suggested (Iceberg requires a catalog — add Nessie? [Yes / Use Hive Metastore instead / I'll pick]).

Conflicts surface immediately. If the user adds Trino 430 after adding Iceberg 1.5, the cart shows: Conflict: Trino 430 is incompatible with Iceberg 1.5 due to REST catalog API mismatch. Switch to Trino 440 (latest compatible) [One click resolution]. The user can override with explicit acknowledgement if they really want a known-broken combination, but this requires confirmation.

7.4 Step 4: Checkout

When the cart is valid (all green), the user proceeds to checkout. They name the stack (production-analytics, fintech-dev, etc.), describe its purpose for future reference, and provide server connection details: SSH credentials for VM targets, kubeconfig path for Kubernetes targets, or cloud credentials if requesting platform-managed infrastructure provisioning.

7.5 Step 5: Inspect Server

The Inspector kicks off immediately. The Console shows a live dashboard as the agent probes the target environment. Within 30 seconds typically, the user sees: CPU model and cores, RAM available, disk per mount with free space and IOPS, OS family, kernel version, installed Java/Python/Docker versions, network reachability, and any pre-existing services that might conflict.

If inspection fails (bad credentials, network unreachable, target server too small), the failure is reported with a specific diagnostic and remediation suggestion. The user can correct and retry without going back to the start.

7.6 Step 6: Allocate Resources

Once inspection completes, the Resource Sizer produces recommended allocations. The Console shows sliders for each component, pre-set to recommended values, with the inspected total resources displayed as a budget bar. The user can adjust to taste; if allocations exceed available resources, the corresponding slider turns red and the install button is disabled until corrected.

7.7 Step 7: Pre-Flight Check

Before any installation begins, a comprehensive dry-run runs. This validates: every component's ports are available on target nodes, every required external dependency is reachable, sufficient disk space exists, no conflicting services are running, network paths between components are correct, and the agent has all needed permissions. The pre-flight produces a green/yellow/red report; any red items must be resolved before installation.

7.8 Step 8: Install

The user clicks Install with confidence. The Install Console opens, showing a live-streaming view of progress. Each component has a status indicator (pending, installing, verifying, done, failed) and a collapsible log panel. The currently-active task's panel auto-expands. Failed tasks halt the workflow until the user chooses retry, skip (rare), or abort.

Behind the scenes, the controller is executing the install DAG, dispatching tasks to the agent, receiving streaming logs, and updating the Console via WebSocket. Each task is idempotent; retries are safe.

7.9 Step 9: Verify Components

Once installation tasks complete, the platform runs health checks against every component: query engine answers a trivial query, catalog responds to metadata requests, object storage accepts a read/write, orchestrator scheduler is running. Any health check failure here is unusual but possible if the install task technically succeeded but the component is misconfigured. The platform reports the specific failing health check and offers diagnostic options.

7.10 Step 10: Load Demo Data

A synthetic demo dataset is loaded into the lakehouse. For the V1 default stack, this is a 100,000-row healthcare claims dataset — chosen for its realistic schema complexity (joins, dates, money, categorical fields) and Indian healthcare relevance. The user can opt out of demo data or substitute their own sample dataset.

Demo data loading exercises the full stack end-to-end: tables are created in the catalog, files are written to object storage in the chosen format (Iceberg/Hudi/Delta), and the orchestrator schedules a sample DAG. This is the platform proving the stack actually works, not just that the components are running.

7.11 Step 11: Smoke Test Query

A pre-defined smoke test query runs end-to-end. For the healthcare claims demo data, the query might be: `SELECT claim_status, COUNT(*) FROM claims GROUP BY claim_status`. The query exercises the query engine, catalog lookup, object storage read, and format-specific scan operations. Success means the entire pipeline works.

Query results are displayed to the user as proof of life. This is the moment the user knows: My lakehouse works.

7.12 Step 12: Success!

The Console transitions to the post-install state, showing URLs for every component: Trino UI (or equivalent for chosen query engine), Grafana dashboards, Airflow scheduler, the lakehouse query interface. Connection strings for BI tools and ODBC clients are surfaced. The user can immediately start using the lakehouse for their own workloads.

7.13 Step 13: Day-2 Operations Begin

Behind the scenes, ongoing operations have already started. Monitoring dashboards are live. Backup schedules are running (configurable by the user but defaulting to daily for metadata, weekly for object storage). Alert rules are wired — the user gets notified if any component goes unhealthy. The Console moves into Operate mode for this stack; all subsequent visits show health and operations, not setup.

The platform notifies the user when compatible upgrades become available, when usage trends suggest scaling actions, and when DR tests pass or fail. The lakehouse is no longer just installed; it is being operated.

8. The Install Workflow as a State Machine

The install workflow is the most complex piece of the platform. It is implemented as an explicit state machine in the Orchestration Layer's Install Orchestrator, with every transition logged and every state recoverable. The diagram below shows the state machine in full.

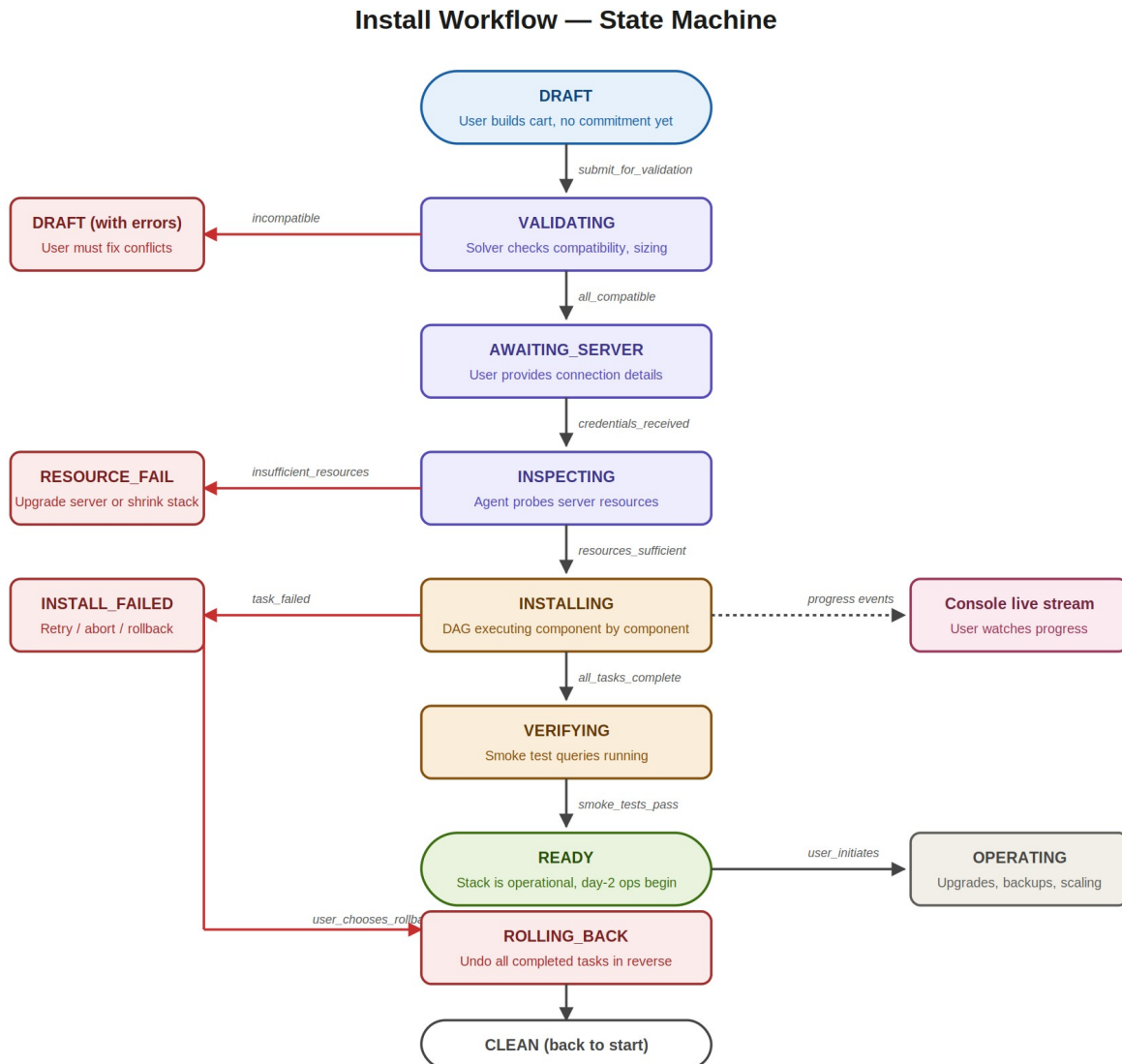


Figure 8 — Install workflow state machine with happy path, failure modes, and rollback.

8.1 The Happy Path

In the happy path, an install moves through these states: DRAFT (user building cart), VALIDATING (solver checking compatibility), AWAITING_SERVER (waiting for credentials), INSPECTING (agent probing target), INSTALLING (executing DAG), VERIFYING (smoke tests running), READY (operational). Each transition is event-triggered and logged.

8.2 Failure Modes and Recovery

Every state has defined failure transitions and recovery paths:

8.2.1 VALIDATING Failures

If the solver determines that the cart contents are incompatible, the workflow transitions to DRAFT with errors. The cart shows the specific conflicts and suggested remediations. No infrastructure is touched.

8.2.2 INSPECTING Failures

If the agent cannot connect to the target (bad SSH credentials, network unreachable, certificate expired), or if the inspected resources are insufficient (less RAM than minimum required by the stack), the workflow transitions to RESOURCE_FAIL. The user can either upgrade the server, shrink the stack, or change targets. No installation has begun.

8.2.3 INSTALLING Failures

If a task in the install DAG fails (component fails to start, port conflict, disk full mid-install), the workflow transitions to INSTALL_FAILED. The user is given three options:

- Retry: re-run the failing task; safe because tasks are idempotent
- Skip: mark the task as not-applicable and proceed (only for non-critical tasks)
- Rollback: undo all completed tasks in reverse order, returning the server to its pre-install state

8.3 The Rollback Path

Every install task registers a rollback hook when it succeeds. When the user chooses rollback, the orchestrator executes rollback hooks in reverse topological order. Components are stopped, configuration files are removed, packages are uninstalled, databases are dropped, and state is cleaned. The state machine moves through ROLLING_BACK and eventually returns to CLEAN.

Rollback is itself idempotent and resumable. If the rollback process fails partway, the user can retry; the orchestrator picks up from where it left off.

8.4 The DAG Structure

The install DAG is computed by the Install Orchestrator from the resolved stack spec. For a typical Iceberg + Trino + Nessie + MinIO + Airflow + Grafana stack, the DAG looks like:

7. Install prerequisites (Java, Python, Docker runtime as needed)
8. Provision MinIO and create buckets (parallel with step 3)
9. Provision Postgres for Nessie and Airflow metadata (parallel with step 2)
10. Install Nessie (depends on Postgres)
11. Register Iceberg tables in Nessie

12. Install Trino coordinator (depends on Nessie being live)
13. Install Trino workers (parallel; depends on coordinator running)
14. Install Airflow scheduler and webserver (depends on Postgres)
15. Install Airflow workers
16. Install Prometheus and Grafana (parallel with everything else)
17. Wire Grafana data sources to Prometheus and Trino
18. Import pre-built Grafana dashboards for the stack
19. Load demo dataset into Iceberg
20. Run smoke test queries
21. Mark stack READY

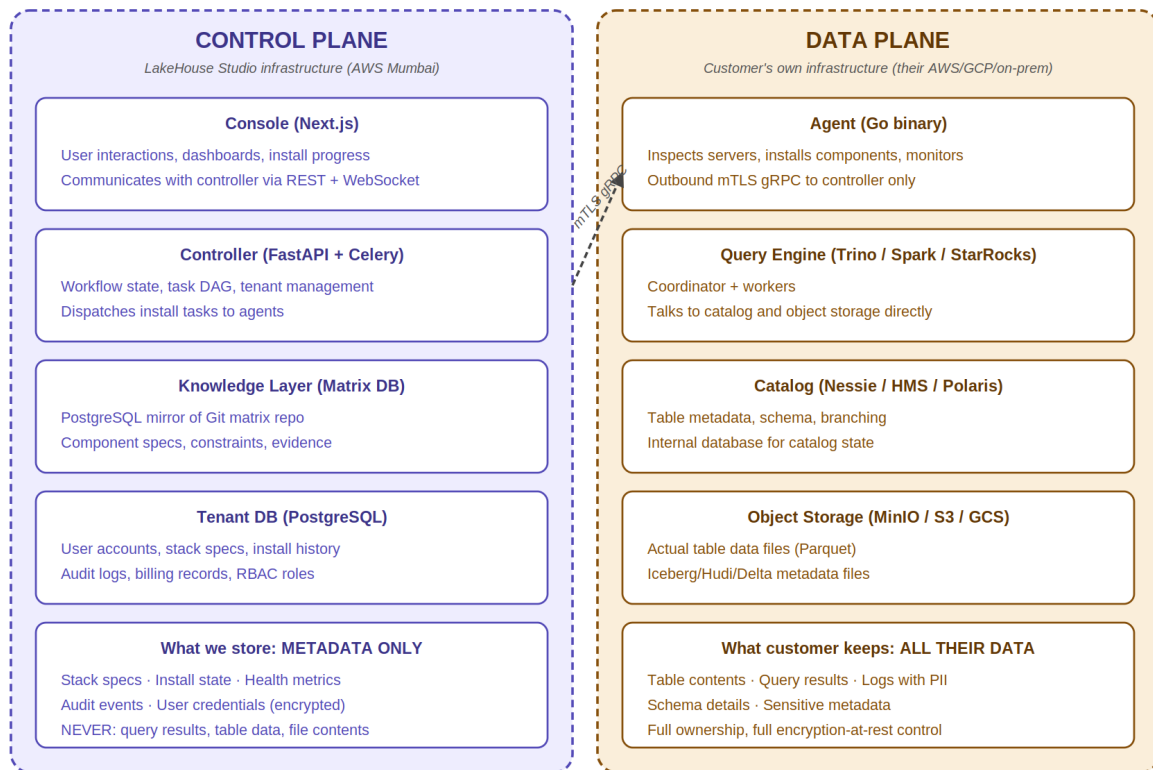
8.5 Concurrency and Parallelism

Tasks with no inter-dependencies run concurrently. In the example above, MinIO and Postgres can install in parallel; Prometheus and Grafana can install in parallel with the data layer. This shortens total install time significantly — typical V1 installs complete in 15-30 minutes on a single VM, despite involving ~15 distinct components.

9. Data Flow and Security Architecture

Security is not a feature of LakeHouse Studio; it is a defining constraint that shaped the entire architecture. This section specifies how data flows, how the control plane and data plane are separated, and the layered security controls that protect both customers and the platform.

Control Plane vs Data Plane — The Critical Separation



*This separation enables DPDP Act 2023 compliance for Indian healthcare, GDPR for EU, and air-gapped deployments.
Customers can self-host the control plane too for ultimate sovereignty.*

Figure 9 — Control plane and data plane separation.

9.1 Control Plane vs Data Plane

LakeHouse Studio enforces a strict separation between control plane and data plane. The control plane (Console, Controller, Solver, Matrix, Tenant DB) handles metadata only: stack specifications, install state, health metrics, audit logs, user accounts. The data plane (Query Engine, Catalog, Object Storage, actual table data, query results) lives entirely within the customer's infrastructure.

This is not a soft separation enforced by policy; it is a hard separation enforced by protocol. The gRPC service definitions between agent and controller have no fields capable of carrying table

data or query results. Health metrics report aggregates (number of queries per minute, average latency, error rates) but never query texts or result rows.

9.2 Layered Security Controls

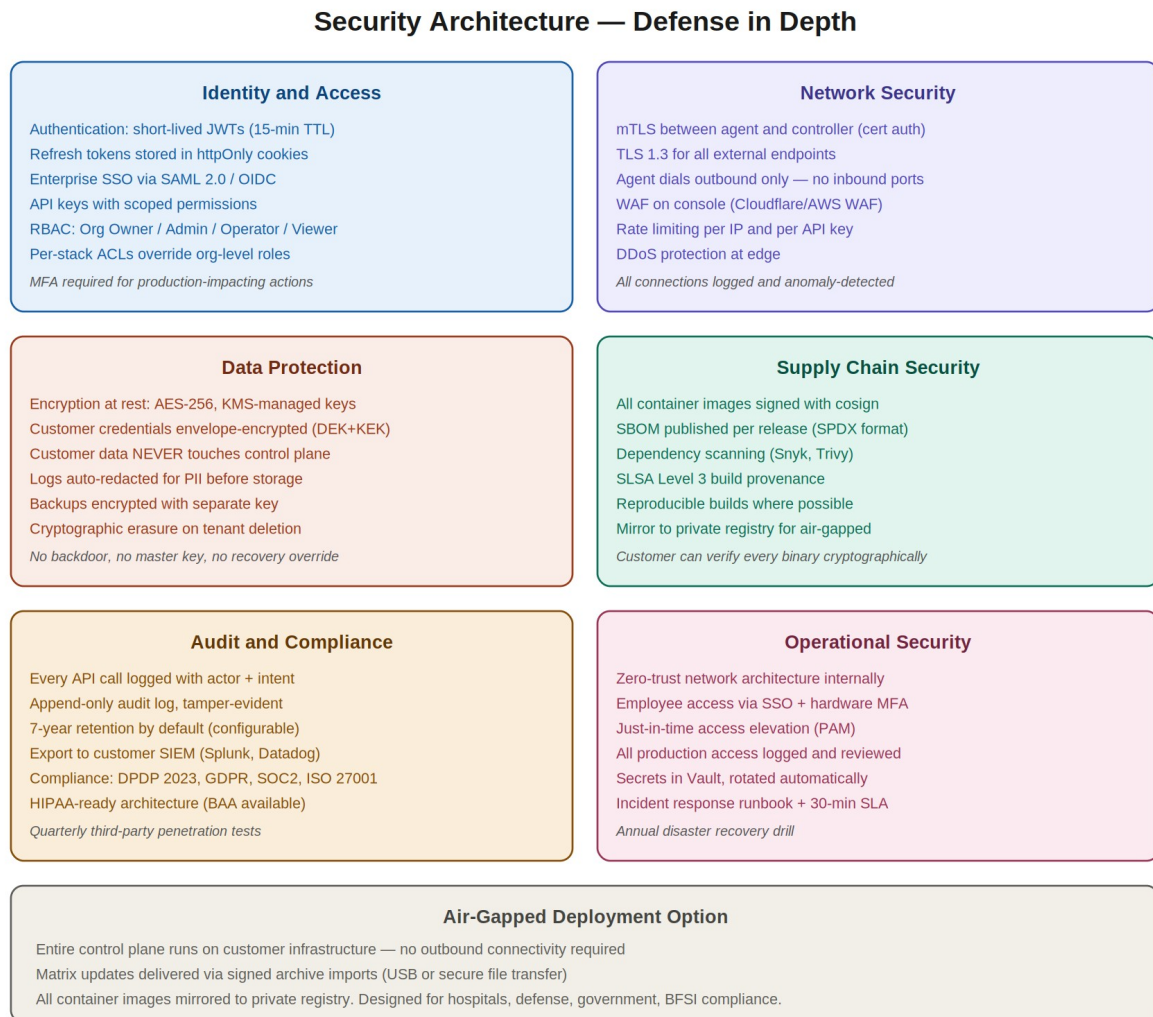


Figure 10 — Defense in depth across six security domains.

9.2.1 Identity and Access

Authentication uses short-lived JWTs (15-minute TTL) for users, refresh tokens stored in httpOnly cookies, and certificate-based authentication for agents. Enterprise SSO is supported via SAML 2.0 and OIDC. API keys are scoped — each key can only access specific resources and operations. RBAC has four built-in roles (Owner, Admin, Operator, Viewer) plus per-stack ACLs for finer control. MFA is required for production-impacting actions.

9.2.2 Network Security

All connections use TLS 1.3. Agent-controller communication uses mutual TLS — both sides authenticate with X.509 certificates. The Console sits behind a WAF (Cloudflare or AWS WAF) with DDoS protection. Rate limiting applies per IP and per API key. The agent dials outbound only; no inbound ports are required, making the platform compatible with strict corporate firewalls.

9.2.3 Data Protection

Encryption at rest uses AES-256 with KMS-managed keys. Customer credentials (SSH keys, kubeconfig contents) are envelope-encrypted: a data encryption key (DEK) encrypts each credential individually, and the DEKs are themselves encrypted with a master key encryption key (KEK) held in KMS. Logs are auto-redacted for PII before storage — patterns for emails, phone numbers, credit cards, and configurable custom patterns are applied at the agent level before logs leave the customer environment.

Critically: customer data never reaches our infrastructure. There is no master key that could decrypt customer data because we have no customer data. Tenant deletion is cryptographic erasure — we destroy the encryption keys, and the encrypted state becomes unrecoverable.

9.2.4 Supply Chain Security

All container images are signed with cosign and verified at deployment. Every release publishes a Software Bill of Materials (SBOM) in SPDX format. Dependency scanning runs continuously via Snyk and Trivy. Build provenance is SLSA Level 3, meaning builds are reproducible and tamper-evident. Customers can verify every binary cryptographically before running it.

9.2.5 Audit and Compliance

Every API call is logged with actor, action, and intent. The audit log is append-only and tamper-evident (hash-chained). Default retention is 7 years, configurable per tenant. Audit logs can be exported in real time to customer SIEMs (Splunk, Datadog, Elastic). Compliance certifications include DPDP Act 2023 (India), GDPR (EU), and SOC 2 Type II (target for Year 2). The architecture is HIPAA-ready and a Business Associate Agreement (BAA) is available on request.

Quarterly third-party penetration tests are performed. Findings are publicly disclosed (after remediation) as part of our commitment to transparency.

9.2.6 Operational Security

Internally, the platform uses a zero-trust network architecture. Employees access production only via SSO with hardware MFA (YubiKey or equivalent). Just-in-time access elevation through a Privileged Access Management (PAM) system means standing production access is zero. All production access is logged and reviewed weekly. Secrets are stored in HashiCorp Vault with

automatic rotation. An incident response runbook with a 30-minute internal SLA governs how we react to issues, with annual disaster recovery drills.

9.3 Air-Gapped Deployment

For the most regulated customers, the entire LakeHouse Studio control plane can be self-hosted within their environment with zero outbound connectivity required. Air-gapped deployment includes:

- All container images mirrored to a customer-controlled private registry
- Matrix updates delivered via signed offline archive imports (USB or secure file transfer)
- Component binaries (Iceberg, Trino, etc.) pre-fetched and stored in the customer registry
- Telemetry disabled; no usage data flows back to LakeHouse Studio
- Self-hosted certificate authority for mTLS between agent and controller

This is designed specifically for hospitals, defense contractors, government agencies, and BFSI customers in jurisdictions that prohibit any cloud-managed control plane.

9.4 Privacy by Design

The platform applies privacy-by-design principles throughout: data minimisation (we collect only metadata necessary for operation), purpose limitation (data collected for one purpose isn't used for another), storage limitation (logs and metrics have defined retention with automatic deletion), and user agency (users can export their data, delete their account, and access full audit trails of what was done with their data). These principles satisfy DPDP, GDPR, and similar regulatory frameworks.

10. Deployment Topology Examples

LakeHouse Studio supports a wide range of deployment topologies. The platform doesn't prescribe one; the user picks based on their constraints (cost, scale, availability needs). Below are three reference topologies that cover most real-world use cases. The diagram shows all three.

Deployment Topology Examples

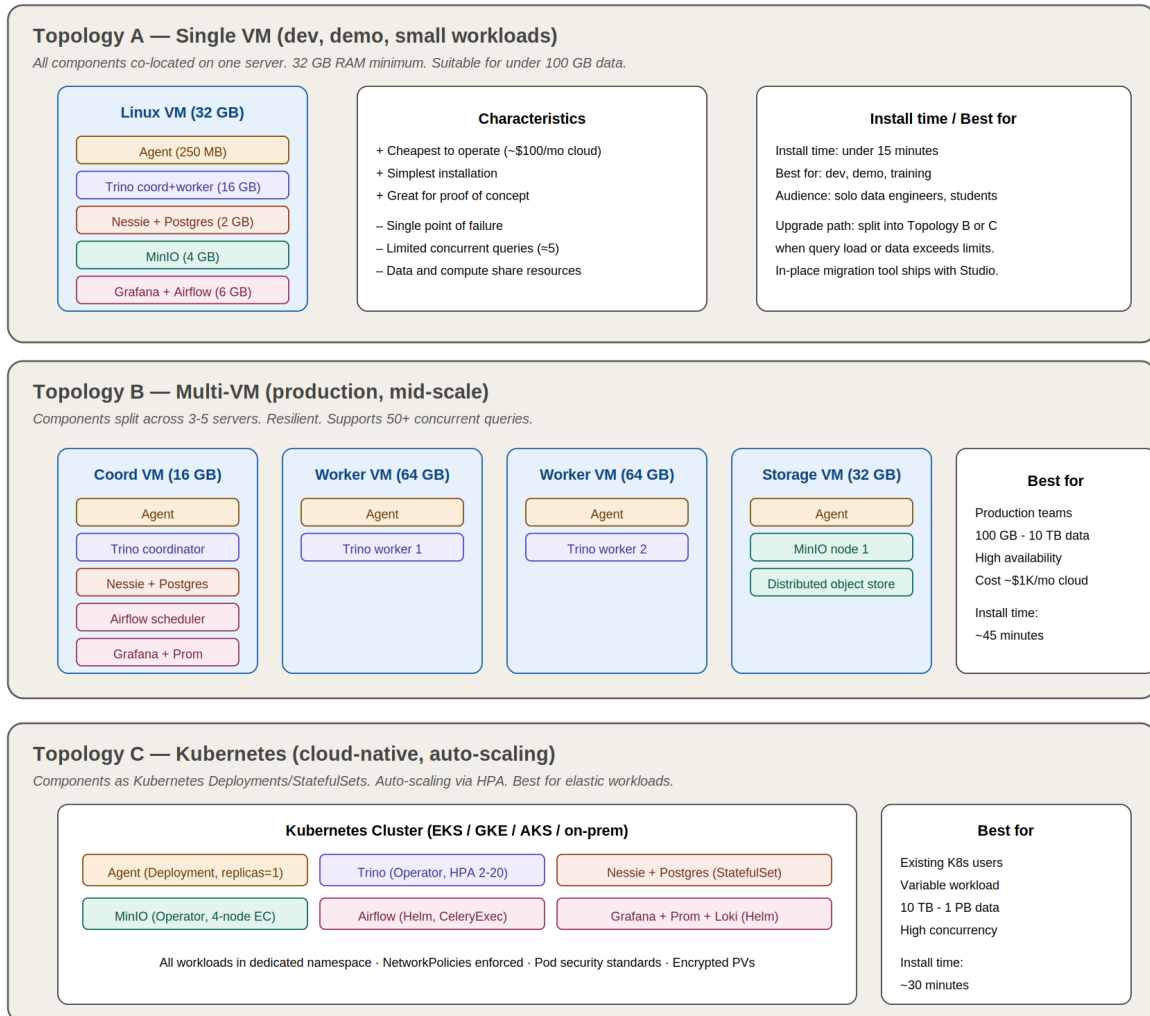


Figure 11 — Three reference deployment topologies for V1.

10.1 Topology A: Single VM

All components co-located on one server. Suitable for development, demos, proof-of-concept work, and small production workloads (under 100 GB data, fewer than 5 concurrent queries).

10.1.1 Resource Requirements

- 32 GB RAM minimum (64 GB recommended)
- 8 CPU cores minimum
- 500 GB disk for typical use
- 1 Gbps network

10.1.2 Component Placement

- Agent: 250 MB RAM
- Trino coordinator + worker (combined): 16 GB RAM
- Nessie + Postgres: 2 GB RAM
- MinIO: 4 GB RAM
- Grafana + Prometheus + Airflow: 6 GB RAM
- System overhead: 4 GB RAM

10.1.3 Trade-offs

Single VM is the cheapest to operate (~\$100/mo on cloud, free on existing on-prem hardware) and simplest to install (15 minutes typical). It is great for proof of concept. The trade-offs are real: single point of failure, limited concurrent query capacity, and data + compute share resources. As workloads grow, users should migrate to Topology B or C; the platform provides in-place migration tooling.

10.2 Topology B: Multi-VM

Components split across 3-5 servers for resilience and scale. Suitable for production workloads handling 100 GB to 10 TB of data with dozens of concurrent users.

10.2.1 Recommended Layout

- Coordinator VM (16 GB): Trino coordinator, Nessie, Postgres, Airflow scheduler, Grafana, Prometheus
- Worker VM 1 (64 GB): Trino worker
- Worker VM 2 (64 GB): Trino worker
- Storage VM (32 GB): MinIO node 1, distributed object store
- Optional Worker VM 3+ for higher concurrency

10.2.2 Trade-offs

Multi-VM provides high availability (any single node failure doesn't bring down the lakehouse), scales linearly with added workers, and isolates resource contention. Costs scale linearly too (~\$1K/mo on cloud for this configuration). Install time is ~45 minutes; ongoing operational overhead is moderate. This is the default recommendation for production deployments.

10.3 Topology C: Kubernetes

Components run as Kubernetes Deployments, StatefulSets, and Operators in a dedicated namespace. Suitable for organisations already invested in Kubernetes who want auto-scaling, declarative configuration, and self-healing.

10.3.1 Recommended Layout

- Agent: Deployment (replicas=1) in lakehouse-studio namespace
- Trino: Trino Operator manages coordinator + workers, HPA scales workers 2-20
- Nessie: StatefulSet with persistent volume for catalog data
- Postgres: StatefulSet (or external managed service like RDS)
- MinIO: MinIO Operator manages 4-node erasure-coded cluster
- Airflow: Helm chart with CeleryExecutor or KubernetesExecutor
- Grafana + Prometheus + Loki: Helm chart (kube-prometheus-stack)

10.3.2 Trade-offs

Kubernetes gives elastic scaling for variable workloads, self-healing through controller reconciliation, and declarative configuration via GitOps. It requires existing K8s expertise and infrastructure; not a great target for teams who don't already use Kubernetes. Install time is similar to multi-VM (~30 minutes) but operational sophistication is higher.

10.4 Migration Between Topologies

Users can start with Topology A and migrate to B or C as workloads grow. The platform provides migration tooling that:

22. Provisions the new topology in parallel with the existing one
23. Replicates catalog metadata to the new environment
24. Mirrors object storage data (or shares the same backend)
25. Switches DNS / load balancer to the new environment
26. Decommissions the old environment after verification

This is one of the platform's strongest features: users don't have to make irreversible architecture decisions upfront.

11. Technology Stack and Justification

Every technology choice in LakeHouse Studio has a justification. This section documents the choices and the reasoning. When alternatives exist, the rejected options are listed with the reason for rejection.

11.1 Console: Next.js 14 + TypeScript + Tailwind + shadcn/ui

Next.js 14 was chosen because React Server Components give fast initial page loads (critical for the Catalog and Inspector views which have large data payloads), the App Router supports both SSR and SPA modes cleanly, and the Vercel-led ecosystem is mature with strong TypeScript support throughout. Tailwind CSS provides utility-first styling that scales across many small components without the maintenance burden of custom CSS classes. shadcn/ui provides accessible, customisable components without the lock-in of a monolithic UI library. TypeScript end-to-end catches type errors at compile time.

Rejected alternatives: Remix (smaller ecosystem, less mature for our use case), plain React + Vite (more setup work, fewer batteries), Vue or Svelte (smaller talent pool in Hyderabad).

11.2 Controller: Python 3.12 + FastAPI + Celery + Redis + PostgreSQL 16

Python was chosen for several reasons. The team has deep Python expertise. The matrix engine's graph algorithms benefit from Python's ecosystem (NetworkX, packaging, Pydantic). Ansible, which we use for VM installs, is Python-native. Future ML integrations (the recommender) are easier in Python. FastAPI specifically because it auto-generates OpenAPI documentation, has native async support, and performs excellently (10K+ req/s per worker). Celery is the de facto Python distributed task queue with mature Redis and RabbitMQ backends. PostgreSQL 16 is the gold standard for relational data with JSONB support for flexible schema where needed.

Rejected alternatives: Go (weaker for graph algorithms and ML), Java/Spring (heavier runtime, slower iteration), Node.js/NestJS (weaker for our specific computational needs).

11.3 Agent: Go + Ansible (embedded) + Helm SDK

Go was chosen because the agent must run as a static binary with no runtime dependencies on customer servers. Go produces single-file binaries with no DLL hell or Python version concerns. Go's networking, concurrency (goroutines), and SSH primitives are excellent. The Kubernetes, Terraform, Docker, and HashiCorp ecosystems all chose Go for the same reasons — it is the lingua franca of modern infrastructure tooling. Ansible is embedded for VM installs because it is the most mature idempotent configuration management tool with thousands of well-tested

modules; we don't need to reinvent that wheel. Helm SDK is used for Kubernetes installs because Helm is the de facto package manager and most upstream projects publish Helm charts.

Rejected alternatives: Rust (slower to develop, smaller talent pool for infra), Python (deployment headaches on diverse customer servers), Java (heavy footprint, JVM startup time).

11.4 Solver: Rust + PyO3

Rust was chosen specifically for the solver because compatibility resolution is performance-critical (must run in under 100ms for keystroke-level UI feedback) and benefits from zero-cost abstractions and no GC pauses. Cargo's own resolver is written in Rust for the same reason, and it has set the bar for what dependency resolution should feel like. PyO3 bindings let the Python controller call Rust functions transparently, getting the performance benefit without the operational cost of running a separate microservice.

Rejected alternatives: Python with z3 or python-sat libraries (10-100x slower at our scale), Go (acceptable but slower than Rust for SAT, and Python integration is more awkward), running the solver as a separate microservice (added latency and operational complexity).

11.5 CLI: Go + Cobra

The CLI is in Go for the same single-binary reasons as the agent. Cobra is the standard CLI framework in the Go ecosystem (used by kubect!, helm, hugo, etc.), with mature support for subcommands, flags, completion, and help generation. The CLI shares some code with the agent (API client, config parsing, output formatting) through internal Go packages.

11.6 Matrix Storage: Git + YAML + PostgreSQL mirror

YAML files in a Git repo are the source of truth because they are human-readable, diff-friendly, support full version history, and enable community contribution via familiar pull request workflows. PostgreSQL is the runtime mirror because queries against the matrix during request handling need to be fast (sub-millisecond) and structured queries against a relational schema are vastly more efficient than parsing YAML files on every request. A CI job synchronises Git to Postgres whenever the matrix repo changes; the mirror is always derivable from Git, never the other way around.

11.7 Transport: gRPC + Protocol Buffers + mTLS

gRPC was chosen for agent-controller communication because it supports bidirectional streaming (needed for live log and metric streams), has strong cross-language support (our agent is Go, controller is Python), and provides excellent built-in support for mTLS

authentication. Protocol Buffers give schema evolution (we can add fields without breaking older agents), compact wire format (important for log streaming), and code generation in both languages. mTLS ensures both ends are authenticated cryptographically, which is the right primitive for a trust relationship between control and data planes.

11.8 Observability: OpenTelemetry + Prometheus + Grafana + Loki

Internally, the platform itself uses OpenTelemetry for tracing, Prometheus for metrics, and Loki for logs. These are open standards. For customer-facing observability (the dashboards LakeHouse Studio installs as part of the user's lakehouse), we also use Grafana + Prometheus + Loki because they are open source, widely understood by operations teams, and ship with battle-tested dashboards for every component we support.

11.9 Infrastructure: AWS Mumbai (primary), with multi-region for DR

The hosted control plane runs in AWS Mumbai region. This is deliberate: Mumbai is in India, satisfying DPDP Act 2023 data residency for Indian customers, and AWS Mumbai has the maturity and service breadth we need. Multi-region disaster recovery is to AWS Hyderabad (when available) or AWS Singapore as a fallback. For non-Indian customers, we will offer EU and US regional control planes in subsequent phases.

11.10 Build and Release: Bazel + Cosign + SLSA + GitHub Actions

Bazel provides reproducible builds across the polyglot codebase (Go, Python, Rust, TypeScript). Cosign signs every release artifact — container images, binaries, Helm charts — enabling cryptographic verification. SLSA Level 3 build provenance attests to where and how each artifact was built. GitHub Actions runs the CI/CD pipelines including matrix validation, integration tests, and release publishing.

12. Phased Roadmap (24 Months)

LakeHouse Studio will be built across 24 months in six phases running on four parallel tracks. The diagram below shows the full plan.

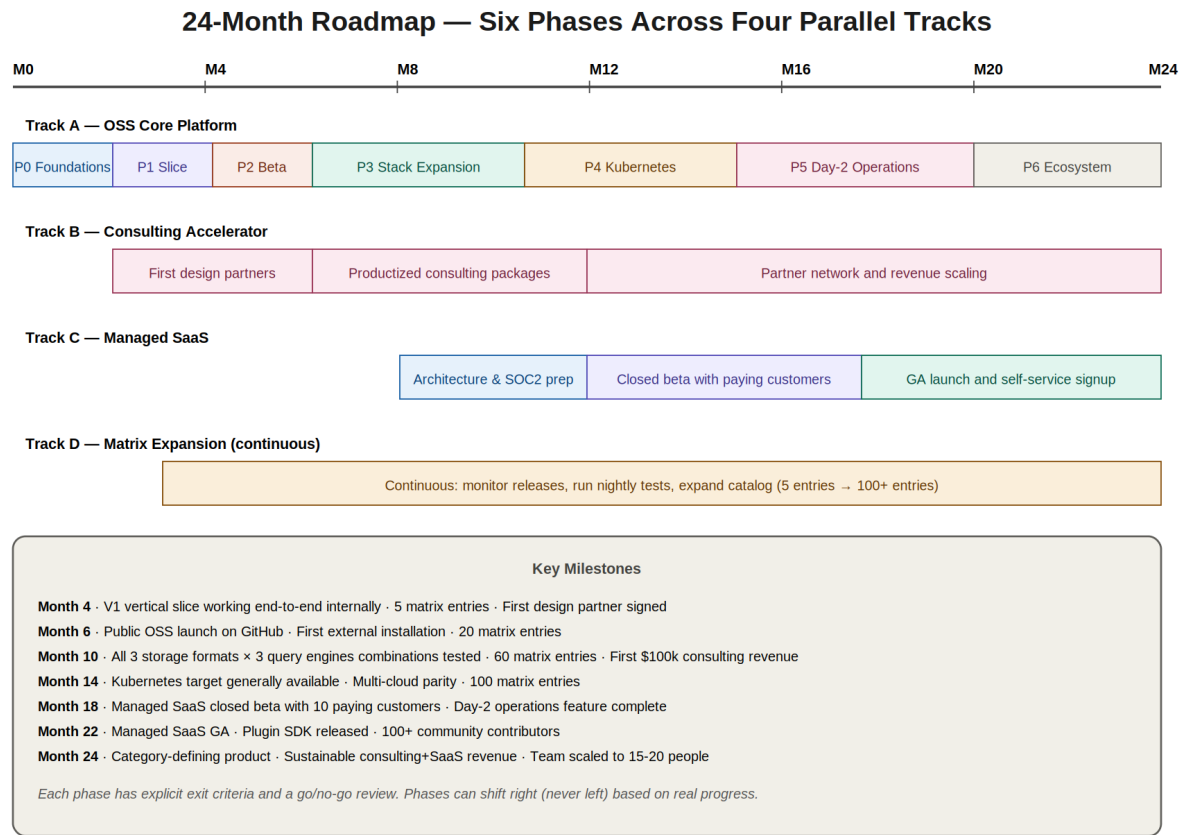


Figure 12 — 24-month roadmap across four parallel tracks.

12.1 The Four Tracks

Track A: OSS Core Platform. The engineering focus. Builds the actual platform that everyone uses. Owned by the core engineering team plus AI tooling.

Track B: Consulting Accelerator. Generates revenue from month four. Uses the early-stage platform to deliver paid implementations to design partner customers. Funds further development. Owned primarily by Vishnu with engineering support.

Track C: Managed SaaS. Deferred to month nine. Builds the hosted commercial offering. Cannot start earlier because we need a stable platform first.

Track D: Matrix Expansion. Continuous from month four. Dedicated team (one engineer plus AI agents) maintains and grows the compatibility matrix.

12.2 Phase Details

12.2.1 Phase 0: Foundations (Months 1-2)

Outputs: Monorepo scaffold, CI/CD pipeline, design system kit, matrix schema specification, agent skeleton with inspection only, controller skeleton with health endpoint. Goal: a developer can clone the repo, run make dev, and have a working local environment.

12.2.2 Phase 1: Vertical Slice (Months 3-4)

Outputs: One stack end-to-end (Iceberg + Trino + Nessie + MinIO on a single Linux VM via SSH); Console with cart and install views (no auth yet, single-user local mode); matrix with 5 entries; agent that can inspect, install, verify. Goal: internal demo that takes a fresh VM to a working query in under 30 minutes.

12.2.3 Phase 2: Multi-Tenant Beta (Months 5-6)

Outputs: Authentication and tenant isolation; hosted controller in AWS Mumbai; five design-partner organisations onboarded; matrix expanded to 20 entries; Airflow added as orchestrator; Grafana monitoring dashboards. Goal: external customers running production lakehouses under DPDP-compliant terms.

12.2.4 Phase 3: Stack Expansion (Months 7-10)

Outputs: Hudi and Delta as storage alternatives; Spark and StarRocks as query alternatives; Dagster as orchestrator alternative; HMS and Polaris as catalog alternatives; permutation testing infrastructure; matrix at 60+ entries. Goal: any of the 54 base combinations actually works.

12.2.5 Phase 4: Kubernetes (Months 11-14)

Outputs: Helm chart generation; operator integration for components that have them; multi-node coordination; persistent volume management. Goal: customers can pick Kubernetes as their deployment target and get the same experience as VM.

12.2.6 Phase 5: Day-2 Operations (Months 15-20)

Outputs: Backup and restore; component upgrades with compatibility re-validation; alerting rules library; RBAC UI; audit log UI; cost estimator. Goal: production lakehouses can be operated for years, not just installed once.

12.2.7 Phase 6: Ecosystem and Polish (Months 21-24)

Outputs: Plugin SDK for community-contributed components; marketplace for community stacks; ML-based stack recommender; public beta of managed offering; partner program. Goal: LakeHouse Studio is a category-defining product with a community and a business.

12.3 Key Milestones

- Month 4: V1 vertical slice working end-to-end internally; 5 matrix entries; first design partner signed
- Month 6: Public OSS launch on GitHub; first external installation; 20 matrix entries
- Month 10: All 3 storage formats x 3 query engines combinations tested; 60 matrix entries; first \$100k consulting revenue
- Month 14: Kubernetes target generally available; multi-cloud parity; 100 matrix entries
- Month 18: Managed SaaS closed beta with 10 paying customers; day-2 operations feature complete
- Month 22: Managed SaaS GA; plugin SDK released; 100+ community contributors
- Month 24: Category-defining product; sustainable consulting+SaaS revenue; team scaled to 15-20 people

12.4 Phase Gates

Each phase has explicit exit criteria. A phase doesn't end on a calendar date; it ends when its exit criteria are met. Phases can shift right (never left) based on real progress. The exit criteria for each phase are:

- Phase 0: Developer can clone repo and run dev environment locally; all CI passes
- Phase 1: Internal install of full stack on fresh VM completes successfully in under 30 minutes
- Phase 2: External design partner installs and uses the platform without our hand-holding
- Phase 3: Permutation test matrix shows green for top-50 combinations
- Phase 4: Kubernetes installs achieve feature parity with VM installs
- Phase 5: A production stack can be backed up, restored, and upgraded with zero data loss
- Phase 6: Managed SaaS has 99.9% uptime SLA across 30 consecutive days

13. Risks and Mitigations

Every significant project has risks; honest risk assessment is part of professional engineering. The risks below are the ones we judge most material to LakeHouse Studio's success, with our mitigation plans.

13.1 Matrix Maintenance Burden

Risk: Iceberg ships releases every 6-8 weeks, Trino every 2 weeks, others on similarly frequent cadences. If matrix maintenance lags, the platform becomes a liability — users encounter combinations the platform claims work but actually don't.

Mitigation: Dedicated team (Track D) from month four; automated upstream release detection via release-feed monitoring; AI-assisted matrix entry drafting; community PR pipeline to distribute maintenance load; nightly re-validation tests catch regressions early; clear deprecation policy when older versions can no longer be supported.

13.2 Component Vendor Changes

Risk: Apache projects can change governance, deprecate features, or shift APIs. Databricks owns Delta Lake, Onehouse owns Hudi, and both have commercial interests that could conflict with ours. Major version bumps can break compatibility broadly.

Mitigation: Matrix tracks not just versions but also feature flags and behavioral compatibility; we publish our test suite publicly so behavioural changes are detectable immediately; deep engagement with upstream communities (we attend conferences, contribute upstream, sit on relevant committees); ability to fork and maintain critical components if necessary (last resort).

13.3 Competitive Pressure

Risk: Databricks, Snowflake, and AWS could build something similar. They have far greater resources and existing customer relationships. Their managed offerings could undercut our pricing.

Mitigation: Speed and focus — we will get to market faster on the specific regulated/on-prem segment they neglect; genuine OSS commitment that big vendors won't match (they always lock in eventually); community moat that takes years to build; deep India-and-emerging-markets focus where big vendors are weaker; the consulting accelerator track gives us direct customer relationships that compound.

13.4 Long Enterprise Sales Cycles

Risk: Hospital and BFSI sales cycles in India routinely take 6-12 months even with great product. Cash runway during this period is precarious.

Mitigation: The consulting accelerator (Track B) generates revenue from month four through direct project work with design partners; consulting engagements convert to longer-term platform contracts; pricing is structured to make initial engagement low-risk for customers (small pilot, expansion if successful); we maintain enough OSS visibility to attract inbound interest, shortening the sales cycle.

13.5 Small Team for Ambitious Scope

Risk: Four humans plus AI augmentation is enough for V1 but stretched thin across four parallel tracks. Burnout and quality erosion are real concerns.

Mitigation: Explicit prioritisation — Track A is the engineering focus, Track B is largely Vishnu's time with light engineering support, Tracks C and D start later when revenue funds hires; AI agents handle well-defined execution tasks (matrix entry drafting, test writing, documentation) to leverage human time; defined phase gates prevent scope creep within a track; we are honest with customers about what is and isn't in V1.

13.6 Open Source Sustainability

Risk: Open source projects often struggle to fund ongoing development. If we don't monetise effectively, the OSS core could stagnate, defeating its strategic purpose.

Mitigation: Clear monetisation path through managed SaaS (Track C); consulting revenue (Track B) funds early-stage development; BSL on the managed control plane code prevents cloud providers from commoditising our SaaS work; we model our economics on successful OSS-plus-managed companies like GitLab, Elastic, MongoDB, and HashiCorp.

13.7 Technical Debt from Speed

Risk: To hit ambitious milestones, we may accumulate technical debt that becomes painful later.

Mitigation: Strict adherence to the architectural principles in Section 3 (these prevent foundational debt); test coverage requirements (75% minimum, 90% for the matrix and solver); architecture decision records (ADRs) documented for every significant choice so future engineers understand context; explicit refactoring slots in the roadmap (10% of engineering time reserved for paying down debt); the layered architecture makes targeted refactoring possible without rewriting everything.

13.8 Security Incident

Risk: A security breach or vulnerability could destroy customer trust irreparably.

Mitigation: Security-first architecture (Section 9); third-party penetration tests quarterly; bug bounty program from launch; coordinated disclosure policy; cyber insurance from a reputable provider; incident response plan with 30-minute internal SLA; transparent communication when issues occur (we publish post-mortems, we don't hide).

13.9 Regulatory Change

Risk: DPDP Act 2023 rules are still evolving in India. GDPR enforcement is becoming stricter. New regulations could require architecture changes.

Mitigation: Architecture designed for sovereignty (control-plane/data-plane separation; self-hostable; air-gappable); legal counsel reviews compliance posture quarterly; close engagement with industry associations (DSCI in India, IAPP internationally); design for regulatory flexibility — we can add new audit fields, data residency options, or consent flows without core rewrites.

13.10 Founder Risk

Risk: A small team's success depends heavily on founder execution and resilience. Burnout, illness, or other disruption could be catastrophic.

Mitigation: Documentation-first culture (this document is an example); decisions are written down so context survives transitions; advisors and mentors engaged from day one; team grows to reduce key-person dependency as quickly as revenue allows; founder welfare actively managed — sustainable pace, mental health support, vacation policy enforced.

14. Glossary and References

14.1 Glossary

Agent. The Go static binary that runs on or near customer infrastructure, performing inspection, installation, monitoring, and lifecycle operations.

Catalog. A component category in the lakehouse stack; stores table metadata (schemas, partitions, locations). Examples: Nessie, Hive Metastore, Polaris.

Cart. The user's pending selection of components in the Console, validated in real time by the solver.

CDCL. Conflict-driven clause learning; a SAT-solving algorithm used by the solver.

Compatibility Matrix. The Knowledge Layer's database of which component versions work together, with evidence and constraints.

Console. The web-based UI built in Next.js. One of three Presentation Layer surfaces.

Controller. The central Python FastAPI application in the Orchestration Layer. Owns workflow state.

Control Plane. The LakeHouse Studio infrastructure that handles metadata, workflow state, and audit logs. Hosted by us (or self-hosted by the customer).

Data Plane. The customer's infrastructure where the actual lakehouse runs. Contains query engine, catalog, object storage, and all customer data.

DAG. Directed acyclic graph; the structure of install task dependencies.

Day-2 Operations. Ongoing operational tasks after initial install: monitoring, backups, upgrades, scaling, troubleshooting.

DPDP Act 2023. India's Digital Personal Data Protection Act, governing personal data handling.

Knowledge Layer. Layer 4 of the architecture; the compatibility matrix.

Lakehouse. A data architecture combining the flexibility of data lakes with the management capabilities of data warehouses, typically using open table formats.

Matrix Entry. A single YAML file describing one component-version, its requirements, compatibilities, and resource profiles.

mTLS. Mutual TLS; both client and server authenticate with X.509 certificates.

Orchestration Layer. Layer 2 of the architecture; the controller.

Presentation Layer. Layer 1 of the architecture; Console, CLI, and Public API.

Resource Sizer. A component in the Intelligence Layer that computes recommended memory and CPU allocations.

Solver. A Rust-based SAT-solving engine that determines which component combinations are compatible.

Stack. A complete set of components installed together, forming one lakehouse instance.

Stack Spec. The fully-resolved specification of a stack with all versions locked and transitive dependencies included.

Target Infrastructure. Layer 6; the customer-controlled infrastructure where the lakehouse actually runs.

14.2 References and Further Reading

Open Table Formats:

- Apache Iceberg: <https://iceberg.apache.org>
- Apache Hudi: <https://hudi.apache.org>
- Delta Lake: <https://delta.io>

Query Engines:

- Trino: <https://trino.io>
- Apache Spark: <https://spark.apache.org>
- StarRocks: <https://starrocks.io>

Catalogs:

- Project Nessie: <https://projectnessie.org>
- Apache Hive: <https://hive.apache.org>
- Apache Polaris: <https://polaris.apache.org>

Orchestrators:

- Apache Airflow: <https://airflow.apache.org>
- Dagster: <https://dagster.io>

Related Architecture Resources:

- The Lakehouse: A New Generation of Open Platforms (Armbrust et al., CIDR 2021)
- Designing Data-Intensive Applications (Martin Kleppmann, O'Reilly)
- Building Evolutionary Architectures (Ford, Parsons, Kua, O'Reilly)
- The Twelve-Factor App: <https://12factor.net>

Compliance and Security:

- DPDP Act 2023 (India): <https://www.meity.gov.in>
- GDPR (EU): <https://gdpr.eu>
- SOC 2: <https://www.aicpa.org>

- SLSA: <https://slsa.dev>

Open Source Governance Models:

- The Apache Way: <https://apache.org/theapacheway>
- CNCF Governance: <https://github.com/cncf/foundation>

14.3 Document Metadata

Version: 1.0

Date: May 2026

Author: Vishnu, Hyderabad, India

Status: Founding technical document

Next review: Quarterly, or upon major scope change

Distribution: Internal team, design partners under NDA, advisors, investors

License: All rights reserved by Vishnu and team. Apache 2.0 license applies to the codebase when released; this architecture document is a confidential business asset until the project is publicly launched.